

Semplificazioni del linguaggio Java (autore: Vittorio Albertoni)

Introduzione

Il linguaggio di programmazione Java, insieme a C e C++ cardine della programmazione per computer con la P maiuscola, non è un linguaggio con cui possa familiarizzare un dilettante.

Esso, disponibile dal 1995, ha una impostazione alquanto difficile, una sintassi parecchio complicata ed è anche molto verboso, spesso costringendo il programmatore a ripetitività.

Per superare i limiti storici di Java, riducendo la verbosità e la quantità di codice ripetitivo (boilerplate) e introducendo funzionalità moderne, sono nati i linguaggi che illustro in questo manuale: Groovy e Kotlin.

Per avere un'idea di che cosa parliamo propongo questi tre modi di scrivere il classico programmino Ciao mondo.

Se utilizziamo il linguaggio Java il codice è il seguente

```
class CiaoMondo
{
    public static void main(String[] args)
    {
        System.out.println("Ciao mondo");
        System.exit(0);
    }
}
```

Se utilizziamo il linguaggio Kotlin il codice è il seguente

```
fun main()
{
    println("Ciao mondo")
}
```

Con il linguaggio Groovy il codice si può ridurre a

```
println("Ciao mondo")
```

Con queste semplificazioni nella scrittura del source arriviamo sempre ad un codice adatto per essere eseguito sulla Java Virtual Machine, come avessimo scritto il source con il linguaggio Java.

Indice

I	Groovy	4
1	Installazione	4
2	Come funziona	5
3	Tipi	6
3.1	Tipi fondamentali	6
3.2	Tipi contenitore	7
4	Variabili e costanti	8
5	Operatori	10
6	Metodi	10
7	Classi	12
8	Packages	13
9	Interattività con l'utente	14
10	Primi esempi di script	15
11	Strutture di controllo	17
12	Altri esempi di script	20
13	Utilità di un IDE	24
13.1	IntelliJ Idea	25
13.2	Eclipse	25
14	Grafica con Groovy	26
14.1	Oggetti contenitori	29
14.2	Colori	31
14.3	Font	32
14.4	Componenti indispensabili per una GUI	33
14.4.1	TextField	33
14.4.2	JLabel	33
14.4.3	JButton	33
14.5	Disegno della GUI	34
14.6	Eventi	35
14.7	Un programma di utilità	35
14.8	Menu	37
14.8.1	JMenuBar	37
14.8.2	JMenu	38
14.8.3	JMenuItem	38
14.9	Qualche altro esempio	39
14.10	Contesto grafico Java in Groovy per il disegno	43
14.10.1	Strumenti per disegnare	43

14.10.2 Realizzare il disegno	44
14.11 Esempio di grafica creativa	45
14.12 Esempio di grafica illustrativa	46
14.13 Animazioni	47
 II Kotlin	 49
1 Installazione	49
2 Come funziona: il primo programma	50
3 Tipi fondamentali	51
4 Variabili e costanti	53
5 Operatori	55
6 Funzioni	56
7 Classi	57
8 Packages	59
9 Interattività con l'utente	61
10 Strutture di controllo	62
11 Integrazione con Java	67
12 Arricchire i programmi con GUI	67
12.1 Oggetti contenitori	68
12.2 Colori	70
12.3 Font	71
12.4 Componenti indispensabili per una GUI	71
12.4.1 JTextField	71
12.4.2 JLabel	72
12.4.3 JButton	72
12.5 Disegno della GUI	72
12.6 Eventi	73
12.7 Un programma di utilità	74
12.8 Menu	76
12.8.1 JMenuBar	76
12.8.2 JMenu	76
12.8.3 JMenuItem	77
12.9 Qualche altro esempio	78
 13 Utilità di un IDE	 82

Parte I

Groovy

Groovy, disponibile dal 2007, nasce da un'idea di James Strachan e Bob McWhirter annunciata nel 2003.

L'idea chiave è creare un linguaggio dinamico che si compili direttamente in bytecode Java, integrandosi nativamente con tutte le librerie ed infrastrutture Java esistenti senza bisogno di layer di adattamento.

Un linguaggio che ha tutte le caratteristiche e la semplicità dei linguaggi di scripting (come Python, Perl, ecc.) e utilizza il grande patrimonio di librerie di Java in maniera che più semplice non si può.

Non possiamo affrontare tranquillamente progetti impegnativi con le semplificazioni del linguaggio Groovy, utilizzando il quale diventa comunque impegnativo fare cose non semplici, ma ci possiamo divertire a produrre script anche non banali.

1 Installazione

Per lavorare con Groovy è necessario che sul computer sia installata la macchina virtuale Java.

Per versioni precedenti la 5.0 non è strettamente necessario che sia installato l'intero Java Development Kit (JDK) e per fare ciò che vedremo in questo manualetto basta il Java Runtime Environment (JRE), quello che dovremmo trovare installato per default in qualsiasi sistema operativo.

Anzi, il compilatore di Groovy può compilare anche codice Java e sostituire il comando `javac` del JDK completo.

Quest'ultimo è comunque d'obbligo se dobbiamo fare building complessi, come creare file `.jar` con il comando `jar` che si trova solo nel JDK completo.

Troviamo Groovy all'indirizzo <https://groovy.apache.org/>.

Nel momento in cui scrivo (settembre 2026) è disponibile la versione stabile 5.1.1 che scarichiamo dalla pagina DOWNLOAD: si tratta di un file compresso in formato `.zip` che, una volta decompresso, ci mette a disposizione, nella directory `bin`, i comandi che servono, sia per sistemi unix like (Linux e Mac) sia per il sistema Windows (comandi con estensione `.bat`), sistema per il quale, per alcune versioni, ci viene fornito anche un installer dedicato.

Per disporre dei comandi stessi in qualsiasi directory siamo collocati dobbiamo sistemare le variabili d'ambiente in modo che la directory dove si trovano i comandi sia nel PATH.

Se lavoriamo su Linux è anche opportuno inserire in `/usr/bin` un collegamento al comando `groovy`, in modo che possa funzionare una shebang standard. Possiamo farlo con il comando

```
ln -s /<percorso al comando groovy da collegare> /usr/bin/groovy
```

Se ci teniamo a disporre della versione più aggiornata, teniamo presente che deve esserci una compatibilità tra le versioni di Groovy e di Java: la versione 5.0.2 di Groovy richiede almeno Java 16 e funziona solo se è installato l'intero JDK.

Chi lavora su una distro Linux legata a Debian (Ubuntu e derivate, MX Linux, Mauna, Mint, ecc.) può trovare una versione non aggiornatissima di Groovy nel repository.

Anche su snap troviamo una versione non aggiornatissima di Groovy.

Installando con queste ultime modalità non abbiamo la versione più recente ma non abbiamo problemi di PATH e collegamenti e dovremmo automaticamente avere compatibilità tra versione di Groovy e versione di Java.

Nella pagina DOCUMENTATION del sito troviamo il manuale ufficiale di Groovy, anche in formato PDF scaricabile.

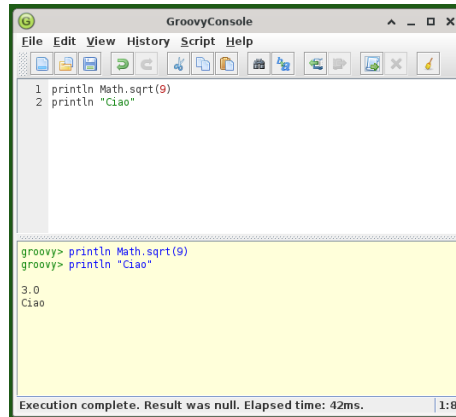
2 Come funziona

Per familiarizzare con Groovy abbiamo a disposizione la console, che si lancia con il comando a terminale

```
groovyConsole
```

o, a seconda di come abbiamo installato Groovy, dal menu delle applicazioni.

Essa si presenta così



Il menu è autoesplicativo e scorrendo il mouse sulle icone della barra degli strumenti otteniamo la descrizione della funzione sottesa a ciascuna icona.

Nella parte superiore dell'area di lavoro scriviamo i comandi e nella parte inferiore dell'area di lavoro vediamo i risultati della loro esecuzione, che avviene cliccando sull'icona .

La parte inferiore può essere ripulita cliccando sull'icona .

Ciò che abbiamo scritto nella parte superiore può essere salvato in un file di script da menu FILE ▸ SAVE AS..., per il quale viene proposta l'estensione facoltativa `.groovy`.

Possiamo poi eseguire lo script con il comando

```
groovy <file_script>
```

Altro modo di comporre lo script è di scriverlo con un qualsiasi editor di testo.

Se lavoriamo su Linux possiamo utilizzare la prima riga dello script per la shebang

```
#!/usr/bin/groovy
```

in modo che, reso eseguibile il file, possiamo lanciarlo scrivendo semplicemente il suo nome.

Fin qui Groovy come linguaggio di scripting, che è materia di questo manuale.

* * *

L'esecuzione dello script nel modo appena visto comporta una breve attesa, in quanto l'interprete deve compilare il bytecode per poi eseguirlo.

Groovy ci offre però un compilatore che produce un file compilato in bytecode, con estensione `.class`, che con il comando `groovy` seguito dal nome del file senza l'estensione, viene eseguito più rapidamente.

La compilazione avviene con il comando

```
groovyc <file_script>
```

e genera un file compilato con lo stesso nome del file di script e con estensione `.class`, come fosse un file compilato java.

Ma non è un file java. E' semplicemente un file che viene eseguito sulla macchina virtuale Java e che, essendo scritto non in codice Java ma in modo semplificato rispetto ad esso, tanto più elevata è questa semplificazione tanto più, per la sua esecuzione, è necessario che nella stessa directory di esecuzione si trovino sia il file script originario sia il file compilato.

Se il file script originario fosse interamente scritto secondo le regole sintattiche del linguaggio Java, potremmo salvarlo con estensione `.java` e compilarlo con il comando

```
groovyc -j <file_java>
```

e il file `.class` generato sarebbe un file Java a tutti gli effetti, pienamente portabile ed eseguibile con il comando `java`, come fosse stato compilato con il comando `javac` del JDK.

Le situazioni intermedie comportano complicazioni che richiedono la profonda conoscenza dei segreti della macchina virtuale Java che non possono essere oggetto di questo manuale, dedicato a Groovy come linguaggio di scripting puro e semplice.

* * *

I comandi che compongono uno script Groovy, oltre che con le librerie proprie `groovy.lang` e `groovy.util`, interagiscono con librerie Java.

Per default sono accessibili senza che sia necessario importarle le librerie

- `java.io.*`
- `java.lang.*`
- `java.math.BigDecimal`
- `java.math.BigInteger`
- `java.net.*`
- `java.util.*`

E' possibile utilizzare qualsiasi altra libreria Java importandola con il comando `import` all'inizio dello script.

3 Tipi

Come tutti i linguaggi di scripting, Groovy si distingue dai linguaggi di programmazione strutturata, tutti a tipizzazione statica, in quanto è un linguaggio a tipizzazione dinamica e il programmatore può decidere se e quando renderlo a tipizzazione statica. Vedremo meglio tutto questo nel prossimo Capitolo.

Per intanto vediamo quali sono i tipi dei dati elaborabili in Groovy.

3.1 Tipi fondamentali

Tipi numerici

Le parole chiave per definire i tipi numerici sono

byte per interi con una dimensione di 8 bit

short per interi con una dimensione di 16 bit

int per interi con una dimensione di 32 bit

long per interi con una dimensione di 64 bit

float per decimali con una dimensione di 32 bit

double per decimali con una dimensione di 64 bit

Per avere un'idea delle grandezze numeriche compatibili con i vari tipi di interi occorre elevare 2 al numero dei bit.

Per i decimali ricordare che il tipo `float` (precisione singola) gestisce fino a 7 cifre decimali significative e il `double` (doppia precisione) gestisce fino a 15 cifre decimali significative.

Queste parole chiave, che con la loro iniziale minuscola sembrano indicare tipi primitivi, come avviene in quasi tutti i linguaggi, in realtà, come avviene in Kotlin, richiamano classi Java, nel caso specifico le classi di `java.lang` con nomi simili con la iniziale maiuscola.

Possiamo peraltro definire i tipi numerici di cui sopra anche usando i nomi delle classi java con la iniziale maiuscola (rammentando che alla parola chiave `int` corrisponde la classe `Integer`).

Per trattare interi e decimali a precisione arbitraria usiamo

BigInteger

BigDecimal

che richiamano le classi omonime del package `java.math`.

Tipi carattere

Per definire i tipi carattere abbiamo

char per un singolo carattere Unicode a 16 bit scritto tra apici semplici o doppi.

String per una stringa di caratteri.

La stringa è una successione di caratteri usata per rappresentare testo.

Il contenuto della stringa normale, immutabile, si scrive tra apici semplici (' e ').

Il contenuto della stringa interpolabile si scrive tra doppi apici (" e ").

In entrambi i casi possiamo utilizzare al loro interno i caratteri di escape, come il new line `\n`.

Introducendo e chiudendo la stringa con tre apici semplici o doppi possiamo scrivere la stringa su più righe e in questo caso non vengono accettati i caratteri di escape.

In entrambi i casi la classe utilizzata è la classe `java.lang.String`.

Tipo booleano

Con

Boolean o **boolean** rappresentiamo i valori logici `true` e `false`.

La classe di riferimento è `java.lang.Boolean`.

3.2 Tipi contenitore

I tipi contenitore sono strutture di dati.

List

La lista è una successione di elementi eterogenei separati da virgola e racchiusi tra parentesi quadre.

Si identifica con la parola chiave

List

Array

L'array è una lista di elementi omogenei del tipo indicato e gli elementi si indicano separati da virgola e racchiusi tra parentesi quadre, come le liste.

Il tipo array si identifica con la sintassi

`<tipo_base>[]`

Con la sintassi

`<tipo_base>[] []`

si identifica una matrice i cui elementi si indicano in una lista contenente le liste che rappresentano le righe della matrice.

In entrambi i casi la classe utilizzata è `java.util.ArrayList`.

Map

E' quello che in altri linguaggi si chiama dizionario o array associativo.

Praticamente è una lista nella quale ogni elemento è costituito dall'associazione tra una chiave e un dato separati dal simbolo :

In questo caso la classe utilizzata è `java.util.LinkedHashMap`.

* * *

Il fatto che Groovy sia un linguaggio a tipizzazione dinamica non vuol dire che esso ignori i tipi, vuole semplicemente dire che le variabili non hanno un tipo fisso. Ma ai valori in quanto tali il linguaggio assegna sempre un tipo per default.

Per verificare il tipo assegnato abbiamo a disposizione la funzione `.getClass()` che è una funzione membro di qualsiasi oggetto Groovy, dove tutto è un oggetto, anche un semplice numero digitato sulla tastiera.

Se nella console di Groovy digitiamo

```
4.getClass()
```

eseguendo otteniamo in risposta

```
Result: class java.lang.Integer
```

Se digitiamo

```
[1,2,3,4].getClass()
```

eseguendo otteniamo in risposta

```
Result: class java.util.ArrayList
```

Come si vede Groovy riconosce la sintassi con cui abbiamo digitato il dato e assegna al dato stesso il tipo previsto per quella sintassi.

Interessante notare come a un numero decimale Groovy assegni per default il tipo a precisione arbitraria `BigDecimal`. Infatti, se digitiamo

```
5.67.getClass()
```

eseguendo otteniamo in risposta

```
Result: class java.math.BigDecimal
```

4 Variabili e costanti

Variabili e costanti, in Groovy, sono oggetti che corrispondono a posizioni di memoria in cui inseriamo un valore per averlo a disposizione per le elaborazioni previste dallo script.

La variabile contiene un valore modificabile, la costante contiene un valore fisso, che non può essere modificato.

Per creare una costante abbiamo a disposizione la parola chiave `final` con questa sintassi

```
final <nome> = <valore>
```

dove <nome> è l'identificativo della costante per poterne richiamare il contenuto quando serve e <valore> è il valore assegnato, che può essere di uno qualsiasi dei tipi visti nel precedente Capitolo.

Con

```
final pi = Math.PI
```

creiamo la costante `pi` contenente il valore di π e ogniqualvolta dobbiamo richiamarlo possiamo farlo scrivendo semplicemente `pi` in luogo di `Math.PI`.

Per creare una variabile possiamo semplicemente usare la sintassi

```
<nome> = <valore>
```

dove <nome> è l'identificativo della variabile per poterne richiamare il contenuto quando serve e <valore> è il valore inizialmente assegnato.

In questo modo la variabile viene creata nel momento in cui le viene assegnato il valore e le viene attribuito il tipo del valore inserito. Se esiste già una variabile con lo stesso nome viene

utilizzata questa e il suo tipo si adegua a quello del nuovo valore: ciò che si chiama tipizzazione dinamica.

E questo è il funzionamento di base di Groovy, funzionamento che alleggerisce il lavoro di programmazione ma che rischia di produrre programmi poco affidabili.

Un primo accorgimento per attutire questo rischio è quello di dichiarare esplicitamente le variabili utilizzando la parola chiave `def`, con la sintassi

```
def <nome>
```

volendo inizializzandola da subito con

```
def <nome> = <valore>
```

Per quanto riguarda la tipizzazione tutto rimane come prima ma la variabile così creata rimane utilizzabile soltanto all'interno della funzione che la contiene. Cioè, mentre la variabile creata senza la parola chiave `def` è una variabile globale, quest'altra è una variabile locale.

Altro contributo alla eliminazione dei rischi della grande libertà che ci è consentita da Groovy è quello del passaggio alla tipizzazione statica, inserendo il tipo della variabile nei comandi per la sua creazione o per la sua dichiarazione, che diventano

```
<tipo> <nome> = <valore>
```

oppure

```
def <tipo> <nome>
```

volendo inizializzandola da subito con

```
def <tipo> <nome> = <valore>
```

Così, la variabile creata

```
int a = 5
```

non potrà che contenere come valore che un numero intero.

Le costanti e le variabili di Groovy sono oggetti e sono dotate di vari metodi. Qui segnalo quelli che ritengo più utili.

metodi delle costanti e delle variabili numeriche

`<nome>.toString()` estrae in tipo `String` il contenuto della costante o della variabile nominata

`<nome>.toInteger()` estrae in tipo `int` il contenuto della costante o della variabile nominata

`<nome>.toFloat()` estrae in tipo `float` il contenuto della costante o della variabile nominata

`<nome>.toDouble()` estrae in tipo `double` il contenuto della costante o della variabile nominata

Ricordare che il casting alla rovescia, da `Double` a `Integer`, comporta la perdita della parte decimale senza arrotondamento.

Interessante notare che, dal momento che Groovy riconosce il tipo adatto per quanto scriviamo sulla tastiera, i metodi che abbiamo visto si possono applicare direttamente a ciò che scriviamo. Per esempio se scriviamo `10.375.toInteger()` otterremo `10`. Se scriviamo `10.toDouble()` otteniamo `10.0`, ecc.

metodi delle costanti e delle variabili stringa

`<nome>.length()` ritorna il numero dei caratteri della stringa nominata

`<nome>.getAt(<indice>)` ritorna il carattere corrispondente all'indice nella stringa nominata

metodi delle costanti e delle variabili lista e array

`<nome>.size()` ritorna il numero degli elementi della lista o array nominati

`<nome>.getAt(<indice>)` ritorna l'elemento corrispondente all'indice (partendo da 0)

`<nome>.add(<elemento>)` aggiunge un elemento alla lista o all'array nominati

`<nome>[<indice>] = <valore>` sostituisce con `<valore>` l'elemento corrispondente all'indice

5 Operatori

Gli operatori collegano tra loro operandi di varia natura in espressioni che forniscono un risultato.

Operatori aritmetici

In ordine di esecuzione sono i seguenti:

- ** per l'elevamento a potenza
- * per la moltiplicazione
- / per la divisione
- % per il modulo (resto della divisione intera)
- + per la somma
- per la sottrazione

Non abbiamo un operatore per la divisione intera che possiamo effettuare utilizzando il metodo `.intdiv()` del dividendo, indicando tra le parentesi il divisore:

`5.intdiv(2)` restituisce 2

Questi operatori operano tra valori numerici.

L'operatore `+` può essere utilizzato per concatenare stringhe.

Operatori di confronto

Servono per confrontare due valori e il risultato che restituiscono è un valore booleano.

Sono i seguenti.

- `==` uguale,
- `!=` non uguale,
- `<` minore,
- `<=` minore o uguale,
- `>` maggiore,
- `>=` maggiore o uguale,
- `===` identico,
- `!==` non identico.

Gli operandi assoggettati al confronto devono avere lo stesso tipo.

Operatori logici

Sono i seguenti.

- `&&` che sta per and,
- `||` che sta per or,
- `!` che sta per not.

6 Metodi

Il metodo, in altri contesti detto anche funzione, procedura o subroutine, è una sezione del programma, identificata con un nome, in cui è racchiusa una serie di istruzioni da eseguire e che vengono eseguite ogni volta che il metodo è chiamato.

Lavorando con Groovy abbiamo a disposizione tutti i metodi contenuti nei packages (librerie) Java.

Nel Capitolo 4 abbiamo visto alcuni metodi ricollegabili agli oggetti costituiti da variabili e costanti. Nella Premessa abbiamo fatto la conoscenza del metodo `println()` per scrivere nello standard output, metodo che si trova nel package `java.io`, importato per default da Groovy.

Nel seguito faremo la conoscenza di altri metodi preconfezionati che Groovy ci mette a disposizione.

Qui vediamo come noi stessi possiamo costruire un metodo, cosa molto utile, per semplificare la scrittura di certi script, quando potrebbe essere utile raggruppare alcune istruzioni all'interno dello stesso script, creando blocchi di istruzioni richiamabili, in modo da suddividere i compiti e da evitare di scrivere più volte le stesse cose.

La sintassi per dichiarare e scrivere un metodo è

```
def <tipo_risultato> <nome> (<tipo_parametro> <nome_parametro>, ....)
{
    <istruzioni>
}
```

dove

<tipo_risultato> è il tipo del risultato che intendiamo ottenere e va indicato nel caso di metodi che ritornino un valore come risultato

<nome> è il nome che diamo al metodo, ed è il nome che useremo per richiamarlo,

<tipo_parametro> e <nome_parametro> sono tipo e nome del parametro (dato) da inserire quando lo richiamiamo, e ce ne possono essere più di uno, separati da virgola,

<istruzioni> sono quelle relative alle elaborazioni da effettuare sui parametri per ottenere il risultato.

Con la parola chiave return possiamo far restituire qualsiasi valore indicato.

La sintassi per chiamare un metodo ed eseguirlo è

<nome_metodo> (<parametri>)

Esempi:

```
def saluta(String nome)
{
    println "Ciao, $nome!"
}
```

L'indentazione non è necessaria ma la utilizzo per migliore chiarezza.

Il metodo richiede l'indicazione del nome della persona da salutare e il saluto avviene interpolando nella relativa stringa il parametro.

Scrivendo

```
saluta("Vittorio")
```

otteniamo

```
Ciao, Vittorio!
```

```
def double radice(double radicando, int indice)
{
    radicando ** (1/indice)
}
```

Sapendo che $\sqrt[n]{x}$ vale quanto $x^{\frac{1}{n}}$ questo metodo calcola radice ennesima di un numero attraverso i parametri radicando e indice di radice.

Scrivendo radice(16,4) otteniamo 2.0.

Scrivendo radice(2,2) otteniamo 1.4142135623730951.

* * *

Quando le istruzioni da svolgere sono in numero limitato e semplici, come avviene per i due esempi appena visti, possiamo ricorrere alla forma abbreviata denominata Closure.

Un closure in Groovy è un oggetto che rappresenta un blocco di codice eseguibile.

Questo blocco di codice può essere assegnato a una variabile, passato come argomento a un metodo o eseguito in un secondo momento richiamandolo come si fa per i metodi. È simile alle funzioni lambda o funzioni anonime in altri linguaggi.

La sintassi per creare un closure è la seguente

```
{<parametri> -> <istruzioni>}
```

Se i parametri sono più di uno vanno indicati uno dopo l'altro separati da virgola.

Utilizzando il closure possiamo creare metodi riutilizzabili assegnando il closure a una variabile.

Per i due esempi precedenti il codice diventa

```
def saluta = {String nome -> println "Ciao, $nome!"}
e
def radice = {double radicando, int indice -> radicando ** (1/indice)}
```

7 Classi

Da ciò che abbiamo visto fin qui, Groovy si presta ottimamente ad un paradigma di programmazione funzionale ma, essendo un linguaggio per oggetti, supporta in pieno la programmazione a oggetti.

Tanti più esperti di me raggiungibili in rete possono ricordare o spiegare al lettore che cosa si intende per programmazione a oggetti.

Io provo a farlo capire con un esempio.

Supponiamo di avere spesso bisogno di determinare area e circonferenza del cerchio di cui ci è noto il raggio.

Se vogliamo evitare di scrivere tutte le volte le necessarie formule, anche perché non è detto che ce le ricordiamo (a volte si ha a che fare con formule che più difficilmente di quelle per calcolare la circonferenza sono rimandabili a memoria), possiamo creare dei metodi che le contengono oppure possiamo ricorrere alla programmazione a oggetti. Se seguiamo quest'altra strada scriviamo una Classe attraverso la quale creare un oggetto Cerchio, contenente metodi (chiamati anche funzioni membro) attraverso le quali calcolare area e circonferenza.

Come si scrive un metodo lo abbiamo visto. Nel caso del nostro esempio il metodo per calcolare l'area del cerchio sarà scritto così:

```
def double area_cerchio (double raggio)
{
    Math.PI * raggio * raggio
}
```

Con

```
area_cerchio(3)
otteniamo
28.274333882308138
```

Se vogliamo seguire i dettami della programmazione a oggetti dobbiamo scrivere la classe Cerchio, dotarla delle funzioni membro per calcolare area e circonferenza, utilizzarla per costruire un oggetto cerchio del raggio desiderato e fare i nostri calcoli richiamando le funzioni membro di questo oggetto.

La sintassi per scrivere la classe è la seguente

```
class Cerchio
{
    double raggio

    Cerchio(double raggio)
    {
```

```

    this.raggio = raggio
}

double area() {Math.PI * raggio * raggio}
double circonferenza() {Math.PI * 2 * raggio}
}

```

Denominata la classe, con iniziale maiuscola come vuole Java, all'interno del blocco di istruzioni indichiamo innanzi tutto i parametri che caratterizzano l'oggetto da costruire (nel nostro caso uno solo, il raggio, espresso in tipo double) poi scriviamo il metodo costruttore e infine i metodi membro che ci servono.

Questa classe può essere utilizzata per costruire un oggetto cerchio avente un certo raggio e dotato dei metodi per calcolarne area e circonferenza in questo modo:

```

mioCerchio = new Cerchio(3)
println mioCerchio.area()
println mioCerchio.circonferenza()

```

Innanzitutto abbiamo costruito l'oggetto mioCerchio come istanza della classe Cerchio con il parametro raggio uguale a 3.

Poi abbiamo scritto di stampare area e circonferenza richiamando i metodi dell'oggetto mioCerchio con raggio 3 per calcolare queste grandezze, con il risultato

```

28.274333882308138
18.84955592153876

```

8 Packages

Il package è una raccolta di funzioni e/o di classi, una libreria.

Il linguaggio Groovy ci mette a disposizione tutti i packages del sistema di sviluppo Java, per fare praticamente tutto ciò che si può immaginare di programmare.

Anche le funzioni e le classi che creiamo noi potremmo inserirle in package, ma non è un lavoro da dilettanti, soprattutto quando si tratta di utilizzarli.

Ai principianti che vogliano conservare librerie prodotte da loro consiglio di tenerle memorizzate in un file di testo e copiare la parte che interessa nel file dove si scrive il programma, lavorando come fatto negli esempi dei capitoli precedenti: in questo modo creiamo pseudo-packages artigianali senza complicazioni.

Ho già elencato nel Capitolo 2 quali packages abbiamo a disposizione per default. Se abbiamo bisogno di altri packages dobbiamo importarli.

L'importazione di tutto ciò che contiene il package si fa con la sintassi

```
import <nome_pacchetto>.*
```

Se ci basta importare una sola classe possiamo farlo con la sintassi

```
import <nome_pacchetto>.<nome_classe>
```

Tra ciò che abbiamo a disposizione per default vedremo nel prossimo Capitolo ciò che ci può servire per creare interattività con l'utente.

Qui merita menzione ciò che ci offre il package java. lang. Math.

Package Math

Contiene costanti e funzioni matematiche che ci possono servire per qualsiasi calcolo possiamo immaginare. Esse si richiamano con la sintassi

```
Math.<costante_o_funzione>
```

Esaminiamole per raggruppamenti.

Costanti

Abbiamo le due più utilizzate costanti matematiche E (numero e, base dei logaritmi naturali), PI (pi greco).

Radici e potenze

`pow(<base>, <esponente>)` ritorna in `double` la base elevata all'esponente indicato, espressi in `double`,

`exp(x)` ritorna in `double` il numero e elevato a x, espresso in `double`,

`sqrt(x)` ritorna in `double` la radice quadrata di x, espresso in `double`,

Per radici ennesime dobbiamo ricorrere alla funzione `pow` indicando come esponente il reciproco dell'indice di radice, dato che $\sqrt[n]{x} = x^{\frac{1}{n}}$.

Funzioni trigonometriche

Tutte le funzioni trigonometriche hanno l'argomento espresso in `double` radianti e ritornano un valore `double`. Le inverse hanno l'argomento in `double` e ritornano un valore in `double` radianti.

Abbiamo le funzioni `sin(x)`, `cos(x)`, `tan(x)` e le funzioni inverse `asin(x)`, `acos(x)`, `atan(x)`.

Abbiamo pure le seguenti funzioni di conversione

`toDegrees(x)` converte radianti in gradi, il tutto espresso in `double`,

`toRadians(x)` converte gradi in radianti, il tutto espresso in `double`.

Funzioni iperboliche

Tutte le funzioni iperboliche hanno l'argomento espresso in `double` e ritornano valori in `double`.

Abbiamo le funzioni `sinh(x)`, `cosh(x)`, `tanh(x)`.

Funzioni varie

Tutte queste funzioni operano su tipi `double`.

`abs(x)` ritorna il valore assoluto di x,

`ceil(x)` ritorna, in `float64`, il più piccolo intero maggiore o uguale a x,

`floor(x)` ritorna, in `float64`, il più grande intero minore o uguale a x,

`round(x)` ritorna il numero intero più prossimo a x,

`log(x)` ritorna il logaritmo naturale di x,

`log10(x)` ritorna il logaritmo decimale di x

`random()` ritorna un numero casuale di tipo `double` compreso tra 0 e 1,

9 Interattività con l'utente

Uno script che si rispetti prima o poi deve interagire con l'utente, quanto meno per evidenziare all'utente stesso i risultati delle elaborazioni previste nello script e, spesso, anche per chiedere all'utente di inserire dati o esprimere scelte per proseguire le elaborazioni.

I metodi che abbiamo a disposizione per queste cose sono i seguenti.

println

Questo metodo scrive l'output a terminale usando una formattazione di default.

Gli elementi da scrivere, che passiamo come parametri, possono essere nomi di variabili (e ne verrà scritto il valore), letterali numerici o espressioni numeriche (e ne verrà scritto il risultato), stringhe (parole o frasi racchiuse tra doppi apici).

Più parametri stringa si inseriscono separati dal simbolo +.

Per iniettare il valore di una variabile in una stringa si può inserire nella stringa il nome della variabile preceduto dal simbolo \$, secondo la tecnica dell'interpolazione della stringa, che, in questo caso, deve essere scritta tra doppi apici.

println inserisce automaticamente un new line e, dopo avere scritto, va a capo.

Nel caso di numeri con molti decimali è possibile limitare la scrittura dei decimali ricorrendo al metodo `.round()` del numero da stampare, indicando tra le parentesi il numero di decimali voluto. Data una variabile `x` contenente il numero 456.6655234426 possiamo stamparne il contenuto con sole tre cifre decimali con

```
println x.round(3)
ottenendo 456.666
```

Esempio:

Supponendo di avere una variabile `a` contenente il valore 12.5, l'istruzione `println("3 moltiplicato 4 fa " + 3*4 + " e la variabile a vale " + a)` scrive a terminale quanto segue

```
3 moltiplicato 4 fa 12 e la variabile a vale 12.5
>>>
```

Lo stesso risultato si sarebbe ottenuto scrivendo

```
println("3 moltiplicato 4 fa " + 3*4 + " e la variabile a vale $a")
```

print

Fa tutto quello che fa `println` ma non inserisce il new line e non va a capo quando ha scritto. Per andare a capo occorre inserire il new line `\n` come stringa.

Non andando automaticamente a capo non è adatto per essere usato nella shell interattiva.

readLine

Questo metodo dobbiamo richiamarlo ricorrendo alla classe `System.in` del package `java.lang` con la sintassi

```
System.in.newReader().readLine()
```

Legge l'input dalla tastiera come stringa e lo può collocare in una variabile con

```
def <nome_variabile> = System.in.newReader().readLine()
```

Per fare in modo che il dato inserito con la tastiera venga letto con tipo diverso dal tipo stringa dobbiamo aggiungere il tipo desiderato con la parola chiave `as`.

```
System.in.newReader().readLine() as int
System.in.newReader().readLine() as double
System.in.newReader().readLine() as BigInteger
System.in.newReader().readLine() as BigDecimal
leggono il dato secondo il tipo indicato.
```

10 Primi esempi di script

Per stilare uno script serve un editor di testo, meglio se pensato per la programmazione, come Geany, che ci consente dal suo interno di lanciarlo e compilarlo.

Piccoli script che non prevedano interattività con l'utente potrebbero essere scritti e provati anche nella console, che è tuttavia più adatta per provare comandi singoli.

Se vogliamo inserire commenti possiamo farlo su una sola riga, anche la stessa di un comando dopo aver scritto il comando stesso, premettendo il simbolo `//`.

Commenti su più righe possono essere scritti tra i simboli `/*` e `*/`.

Se lavoriamo in Linux possiamo utilmente inserire nella prima riga la shebang

```
#!/usr/bin/groovy
```

in modo che lo script, reso eseguibile, sia lanciabile senza ricorrere all'interprete.

Cominciamo con un semplice script che chiede all'utente il nome per salutarlo e che scrivo nella versione più stenografica del linguaggio

```
println 'Come ti chiami?'
nome = System.in.newReader().readLine()
println "Ciao, $nome!"
```

Bene sapere che l'interprete Groovy può eseguire anche script contenenti un misto di linguaggio Java e Groovy e addirittura script interamente scritti in linguaggio Java.

Potremmo comporre questo script, avente la stessa finalità del precedente, con un misto di linguaggio Java e Groovy, così

```
class Saluto
{
    static void main(String[] args)
    {
        String nome
        println("Come ti chiami?")
        nome = System.in.newReader().readLine()
        println "Ciao, $nome!"
    }
}
```

e funzionerebbe.

Potremmo anche comporre lo script interamente in linguaggio Java

```
class Saluto
{
    public static void main(String[] args) throws Exception
    {
        BufferedReader input = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Come ti chiami?");
        String nome = input.readLine();
        System.out.println("Ciao " + nome + "!");
        System.exit(0);
    }
}
```

ed anche così funzionerebbe.

Quest'altro script può servire per calcolare una potenza in precisione arbitraria.

```
print 'Indica la base: '
b = System.in.newReader().readLine() as BigDecimal
print "Indica l'esponente: "
e = System.in.newReader().readLine() as int
```



```
p = b ** e
println p.round(2)
```

Il risultato viene scritto arrotondando alla seconda cifra decimale e può essere un numero di svariate centinaia di cifre.

11 Strutture di controllo

Come tutti i linguaggi di programmazione, Groovy ci offre il modo di controllare l'esecuzione del programma attraverso istruzioni per condizionare l'esecuzione di un blocco al verificarsi di certe condizioni oppure per la ripetizione di blocchi.

if

L'istruzione `if`, chiamata istruzione di esecuzione condizionale (in inglese `if` è il nostro `se`), ci dà modo di assoggettare l'esecuzione di un blocco di istruzioni al verificarsi di una determinata condizione: se la condizione è vera viene eseguito il blocco di istruzioni, altrimenti si prosegue l'esecuzione del programma saltando il blocco stesso.

La sintassi è la seguente

```
if (<condizione>)
{
    <istruzioni>
}
```

dove `<condizione>` è una qualsiasi espressione che relaziona due valori attraverso operatori di confronto: se la condizione si verifica vengono eseguite le istruzioni contenute tra le parentesi graffe, altrimenti si passa oltre.

L'uso delle parentesi graffe è d'obbligo se le istruzioni sono più di una. Nel caso di una sola istruzione essa si può semplicemente scrivere senza parentesi, anche di seguito a `if (<condizione>)`, sulla stessa riga.

L'istruzione `if` si presta anche all'esecuzione condizionale a due rami. Per ottenere questo dobbiamo abbinarla all'istruzione `else` con questa sintassi

```
if (<condizione>)
{
    <istruzioni>
}
else
{
    <istruzioni>
}
```

In questo caso se la condizione si verifica vengono eseguite le istruzioni contenute nel primo blocco altrimenti vengono eseguite quelle contenute nel secondo blocco dopo `else` (altrimenti). In ogni caso proseguendo poi nell'esecuzione del resto del programma.

Possiamo infine gestire l'esecuzione condizionale a più rami abbinando a `if` l'istruzione `else if` con questa sintassi

```
if (<condizione>)
{
    <istruzioni>
}
```

```

else if (<condizione>)
{
    <istruzioni>
else if (<condizione>)
{
    <istruzioni>
}
.....

```

e proseguire quanto vogliamo.

Il seguente script

```

println('Inserisci un numero')
x = System.in.newReader().readLine() as int
if (x < 10) println 'Ci sei dentro'
else println('Ciao')

```

se inseriamo un numero minore di 10 stampa la frase «Ci sei dentro»; se inseriamo 10 o un numero superiore stampa il saluto «Ciao».

Quest'altro

```

println('Inserisci un numero')
x = System.in.newReader().readLine() as int
if (x < 10) println('Sei dentro')
else if (x >= 10 && x <= 12) println('Sei appena fuori')
else println("Sei completamente fuori")
println("Ciao")

```

se inseriamo un numero minore di 10 stampa la frase «Ci sei dentro» e poi il saluto «Ciao»; se inseriamo 10 o 11 o 12 stampa la frase «Sei appena fuori» e poi il saluto «Ciao»; se inseriamo un numero superiore a 12 stampa la frase «Sei completamente fuori» e poi il saluto «Ciao».

switch

Il costrutto switch è un altro modo di fare ciò che possiamo fare con la serie di else e else if.

Con questo costrutto l'ultimo esempio che ho fatto può essere scritto così

```

println('Inserisci un numero')
n = System.in.newReader().readLine() as int
switch (n)
{
    case {n in 0..9}-> println('Sei dentro')
    case {n in 10..12} -> println('Sei appena fuori')
    case {n > 12} -> println('Sei completamente fuori')
}
println 'Ciao'

```

Come si vede, si abbinano direttamente istruzioni semplici ai valori che può assumere una determinata variabile, che può anche essere una stringa, come vediamo nel seguente esempio.

```

println('Inserisci il colore del semaforo con lettera minuscola')
semaforo = System.in.newReader().readLine()
switch (semaforo)
{
    case "rosso" -> println "Fermati"

```

```

    case "verde" -> println "Passa tranquillo"
    case "giallo" -> println "Rallenta perché dovrai fermarti"
    default -> println "Dovevi inserire uno dei colori del semaforo in minuscolo"
}

```

In questo caso, a seconda del colore che scriviamo, abbiamo la risposta che ci indica come comportarci e se inseriamo una parola che non indica un colore del semaforo o se non usiamo l'iniziale minuscola come richiesto abbiamo l'invito a comportarci come si deve.

while

Si usa quando si vuole ripetere l'esecuzione di una istruzione o di un blocco di istruzioni fino a quando rimane vera (true) una determinata condizione booleana.

E' la tipica istruzione che consente di ripetere l'esecuzione un numero definito di volte utilizzando un contatore.

La sintassi per l'uso di questa istruzione è

```

contatore = 1
while (contatore <= n)
{
    <istruzioni>
    contatore = contatore + 1
}

```

La variabile che ho battezzato `contatore` per richiamarne la natura, viene inizializzata con il valore 1. Ad ogni esecuzione delle istruzioni viene aumentata di 1 e, una volta raggiunto un valore `n` voluto, si ferma l'esecuzione. In genere viene nominata semplicemente `i`.

Questo piccolo script

```

i = 1
while (i <= 3)
{
    println("Ciao, Vittorio")
    i = i + 1
}

```

scrive la frase Ciao, Vittorio per tre volte.

Quest'altro

```

i = 1
while (i <= 6)
{
    println(i)
    i = i + 1
}

```

elenca i numeri da 1 a 6.

In entrambi gli esempi il blocco da ripetere contiene una sola istruzione oltre a quella di aumentare di 1 il contatore ma, ovviamente, può contenerne quante vogliamo.

for

Si usa quando si vuole ripetere l'esecuzione di una istruzione o di un blocco di istruzioni ogniqualvolta è vera (true) una determinata condizione booleana, verificata attraversando una qualsiasi struttura (stringa, range di valori, ecc.)

La sintassi per l'uso di questa istruzione è

```
<definizione di una struttura>
for (<variabile> in <struttura>)
{
    <istruzioni>
}
```

Se definiamo la struttura come un range di numeri interi (per esempio da 1 a 5) con l'istruzione for possiamo fare ciò che si fa con while.

Per scrivere tre volte il saluto «Ciao, Vittorio!» possiamo fare così:

```
r = 1..3
for (x in r) println('Ciao, Vittorio')
```

La struttura potrebbe essere una stringa e potremmo collegare l'esecuzione di un'istruzione o di un blocco di istruzioni alla presenza di un certo carattere o tipo di carattere contenuto nella stringa, come nei seguenti esempi.

Il seguente script

```
s = "Vittorio"
for (x in s)
{
    if (x == "t")
        println("trovato t")
}
```

scrive due volte la frase «trovato t» avendo trovato due volte la lettera t nella stringa Vittorio.

12 Altri esempi di script

Con le strutture di controllo abbiamo esaurito l'esame degli strumenti che ci mette a disposizione il linguaggio Groovy per creare script eseguibili da riga di comando, cioè senza interfaccia grafica e possiamo fare qualche esempio un tantino più complicato di quelli presentati nel Capitolo 10.

Cominciamo da alcuni script per i quali bastano le conoscenze si qui acquisite, che comprendono metodi per interfacciarsi con l'utente, operatori, metodi matematici di base e strutture di controllo.

Questo script elenca i numeri primi contenuti in un certo intervallo.

```
def sePrimo(n)
{
    if (n <= 1) println("")
    else
    {
        int x = n.intdiv(2)
        while (x > 1)
        {
            if (n % x == 0) break
            x = x - 1
        }
        if (x == 1) println(n.toString())
    }
}

println('RICERCA NUMERI PRIMI IN UN INTERVALLO')
def minore = System.console().readLine('Inizio intervallo ').toInteger()
```

```
def maggiore = System.console().readLine('Fine intervallo ').toInteger()
for (int i = minore; i < maggiore; i = i + 1)
{
    sePrimo(i)
}
```

Lo script definisce il metodo `sePrimo()` che, secondo un semplice algoritmo che non ho inventato io ma ho trovato in rete, verifica se un numero è un numero primo e, se sì, lo stampa. Poi vengono chiesti all'utente gli estremi dell'intervallo di ricerca e il metodo `sePrimo()` viene applicato ai numeri contenuti nell'intervallo.

Quest'altro script calcola il fattoriale di un numero (corrispondente al prodotto di quel numero per il prodotto di tutti i numeri interi che lo precedono) applicando la precisione arbitraria in modo da ottenere e scrivere anche numeri molto grandi, di svariate centinaia di cifre.

```
def BigInteger fattoriale(int n)
{
    if(n==0) 1
    else n * fattoriale(n-1)
}
println 'Inserisci il numero di cui calcolare il fattoriale:'
n = System.in.newReader().readLine() as int
println fattoriale(n)
```

Qui abbiamo un bel metodo ricorsivo per calcolare il fattoriale definito in modo tale che ritorni un `BigInteger`, cioè un intero in precisione arbitraria, generandolo da un normale intero. Poi viene chiesto all'utente di indicare il numero di cui calcolare il fattoriale e viene stampato il risultato dell'applicazione del metodo al numero indicato.

Sull'onda del successo di questo script con metodo ricorsivo per calcolare il fattoriale di un numero viene voglia di provare la stessa tecnica ricorsiva con un altro algoritmo, quello per trovare il numero corrispondente ad una certa posizione nella successione di Fibonacci (quella successione nella quale un numero corrisponde alla somma dei due numeri che lo precedono).

```
def BigInteger fibonacci(int n)
{
    if (n <= 1) return n
    fibonacci(n - 1) + fibonacci(n - 2)
}
println 'Indica la posizione nella successione di Fibonacci'
n = System.in.newReader().readLine() as BigInteger
println "Nella posizione $n della successione di Fibonacci abbiamo il numero:"
println fibonacci(n)
```

Anche qui abbiamo un metodo ricorsivo per calcolare il valore cercato e un dialogo per avere dall'utente indicazione su quale valore cercare.

Visto che la successione di Fibonacci dopo un po', anche se in maniera meno esplosiva del fattoriale, trova numeri di una certa dimensione, lo script prevede la precisione arbitraria utilizzando il tipo `BigInteger`.

Precauzione inutile in quanto il metodo ricorsivo utilizzato è molto inefficiente a causa del grande numero di ricalcoli che comporta il passaggio `fibonacci(n - 1) + fibonacci(n - 2)`. Tanto che dalla trentesima posizione in poi occorre sempre più tempo per ottenere il risultato, fino ad attese impossibili.

Tutto cambia se utilizziamo un normale metodo basato sul ciclo `for`:

```

def BigInteger fibonacci(int n)
{
    if (n <= 1) return n
    BigInteger a = 0, b = 1
    for (int i = 2; i <= n; i = i + 1)
    {
        BigInteger t = a + b
        a = b
        b = t
    }
    return b
}

println 'Indica la posizione nella successione di Fibonacci'
n = System.in.newReader().readLine() as BigInteger
println "Nella posizione $n della successione di Fibonacci abbiamo il numero:"
println fibonacci(n)

```

Con lo scorrimento di valori già calcolati attraverso la variabile t nel ciclo evitiamo ricalcoli ed otteniamo di poter calcolare in un attimo anche i valori in posizioni molto lontane.

Un altro script che potrebbe essere di qualche utilità possiamo dedicarlo al calcolo combinatorio, cioè al calcolo di permutazioni, combinazioni e disposizioni.

Per le permutazioni semplici, visto che esse corrispondono al fattoriale del numero di elementi da permutare, possiamo utilizzare lo script per il calcolo del fattoriale che abbiamo visto prima.

Per Combinazioni e Disposizioni, semplici senza ripetizione, di n numeri presi k a k dobbiamo le seguenti formule

$C(n, k) = \frac{n!}{(n-k)!k!}$ per le combinazioni,

$D(n, k) = \frac{n!}{(n-k)!}$ per le disposizioni,

nelle quali ricorre spesso il calcolo del fattoriale.

Il fatto che la necessità di questo calcolo ricorra spesso nelle formule e comporti una non semplice scrittura di istruzioni per effettuarlo suggerisce di isolare queste istruzioni in un metodo dedicata al calcolo del fattoriale in modo da poter richiamare queste istruzioni in blocco quando serve.

Lo script per calcolare Combinazioni e Disposizioni potrebbe essere così concepito

```

def BigInteger fattoriale(int n)
{
    if(n==0) 1
    else n * fattoriale(n-1)
}

println 'Quanti elementi utilizzi?'
n = System.in.newReader().readLine() as int
println 'Con quanti elementi formi i raggruppamenti?'
k = System.in.newReader().readLine() as int
c = fattoriale(n)/(fattoriale(n-k) * fattoriale(k))
d = fattoriale(n)/fattoriale(n-k)
println("combinazioni:  $c")
println("disposizioni:  $d")

```

Lanciando questo script saremo richiesti di indicare il numero di elementi utilizzati (n), il numero di elementi per i raggruppamenti (k) e in un attimo vedremo il risultato.

* * *

Fin qui abbiamo utilizzato strumenti che sono stati presentati nel manualetto che sto scrivendo.

Tuttavia Groovy ha a disposizione tutte le librerie Java, anche quelle che non sono disponibili per default e che dobbiamo importare se vogliamo utilizzarle, ovviamente conoscendo i metodi che contengono e come usarli.

Per esempio, se volessimo creare uno script per calcolare i giorni che decorrono tra una data e l'altra avremmo bisogno dei metodi della libreria `java.time`, che contiene metodi per elaborare ore, minuti, giorni, anni, ecc., tutto ciò che ha relazione con il tempo (time) e lo script potrebbe essere il seguente

```
import java.time.*
println 'CALCOLIAMO LA DIFFERENZA TRA DATE'
println 'Prima data:'
print 'giorno: '
g1 = System.in.newReader().readLine() as int
print 'mese: '
m1 = System.in.newReader().readLine() as int
print 'anno: '
a1 = System.in.newReader().readLine() as int
println 'Seconda data:'
print 'giorno: '
g2 = System.in.newReader().readLine() as int
print 'mese: '
m2 = System.in.newReader().readLine() as int
print 'anno: '
a2 = System.in.newReader().readLine() as int
data1 = LocalDate.of(a1,m1,g1)
data2 = LocalDate.of(a2,m2,g2)
differenza = Math.abs(data2.toEpochDay() - data1.toEpochDay())
println "La differenza tra le due date è di $differenza giorni"
```

Lo script sarebbe molto più compatto se utilizzassimo un'altra classe java, la classe `Scanner` del package `java.util` che semplificherebbe la lunga parte dello script dedicata all'acquisizione delle due date tra cui calcolare la differenza.

Ho acquisito la differenza tra le date come valore assoluto in modo da far risultare positivo il numero di giorni intercorrenti tra le date anche nel caso che la seconda data indicata sia antecedente alla prima.

Chi conosca i segreti del package `jama`, sviluppato per Java dal National Institute of Standards and Technology, dedicato all'algebra lineare potrebbe produrre il seguente script per risolvere sistemi di equazioni lineari

```
@Grab(group='gov.nist.math', module='jama', version='1.0.3')
import Jama.*
println 'RISOLUZIONE DI UN SISTEMA LINEARE'
println 'Quante equazioni?'
n = System.in.newReader().readLine() as int
i = 1
l = []
while (i <= n)
{
    println "Coefficienti dell'equazione $i:"
    ii = 1
    ll = []
```

```

    r = 1..n
    while (ii <= n)
    {
        println "Coefficiente $ii: "
        ll.add(System.in.newReader().readLine() as double)
        ii = ii + 1
    }
    i = i + 1
    l.add(ll)
}
double[] [] dati_A = l
Matrix A = new Matrix(dati_A)
l = []
ii = 1
ll = []
r = 1..n
while (ii <= n)
{
    println "Termine noto $ii: "
    ll.add(System.in.newReader().readLine() as double)
    l.add(ll)
    ll = []
    ii = ii + 1
}
double[] [] dati_b = l
Matrix b = new Matrix(dati_b)
Matrix x = A.solve(b)
println 'Soluzione:'
x.print(10,3)

```

Come sempre le indentature non servono ma le ho usate per rendere più chiara la struttura dello script.

La strana dicitura che precede l'importazione del package è dovuta al fatto che si tratta di un package che non è nel PATH di Java ed occorre inserirlo per poterlo importare con il comando nella riga successiva.

Molto laboriosa la scrittura del codice per l'acquisizione dei coefficienti delle variabili nelle equazioni e dei termini noti per la costruzione della matrice del sistema e del vettore verticale dei termini noti, secondo la non semplice sintassi prevista da Jama, per poi ottenere la soluzione.

Altrettanto laboriosa e lunga la fase di inserimento dei dati, specialmente se abbiamo molte equazioni nel sistema, ma molto efficiente il ritrovamento della soluzione che avviene in un attimo, anche in presenza di sistemi con molte equazioni, grazie al metodo `solve()`.

Il vettore che contiene la soluzione del sistema è dotato di un metodo `print()` che accetta due parametri di formattazione: il primo può servire per allineare i valori sulla destra nello spazio di caratteri indicato (se non serve allineamento inserire 0), il secondo indica il numero delle cifre decimali da esporre.

13 Utilità di un IDE

Per realizzare semplici script possiamo benissimo farcela con un qualsiasi editor di testo.

Se lo script è complicato dalla presenza di funzionalità non ricorrenti o se utilizziamo librerie java di cui abbiamo conoscenza superficiale è bello ricorrere ad un IDE (Integrated Development Environment) che, se abilitato a Groovy, ci offre la code completion scrivendo con quel linguaggio.

In questo modo possiamo riscontrare l'esatta dicitura di un comando che non ricordiamo, evitare di ricercare regole sintattiche su manuali, avere contezza dei parametri che è necessario inserire richiamando un metodo, ecc.

A questo proposito segnalo un paio di IDE con i quali si programma molto bene in Groovy.

13.1 IntelliJ Idea

Anche la versione gratuita Community è abilitata per default a trattare progetti Groovy e dobbiamo fare questa scelta all'apertura del nuovo progetto.

Se vogliamo evitare di complicarci la vita e vogliamo semplicemente produrre uno script, del tipo di quelli visti in questo manuale, una volta aperto il progetto, cliccando destro sulla voce `src` nella zona di sinistra dell'IDE, dove abbiamo la struttura del progetto, scegliamo **NEW ▸ GROOVY SCRIPT**, indicando poi il nome da dare allo script stesso, cui sarà di default assegnata l'estensione `.groovy`.

In questo modo l'editor è dedicato a Groovy e non appena abbiamo scritto le prime lettere di un comando ce ne viene suggerito il completamento o indicate le scelte da fare per il completamento.

Inoltre, una volta creato un oggetto, scrivendo il nome di quell'oggetto seguito da un punto (`.`) apriamo un menu a discesa che elenca tutti i metodi collegati a quell'oggetto, dandoci modo di scegliere quello che ci serve.

Se lavoriamo con Groovy avendo importato una libreria Java tutto ciò avviene anche per i metodi e gli oggetti della libreria Java importata.

13.2 Eclipse

La versione di Eclipse per Java può essere abilitata a Groovy installando il relativo plugin in questo modo:

- . da menu **HELP ▸ ECLIPSE MARKETPLACE...** aprire il marketplace, verificare che sia installato il plugin **Eclipse Marketplace Client** e se non è installato installarlo,
- . attraverso la finestra **FIND** ricercare la parola `groovy` e installare il plugin **Groovy Development Tools** che comparirà come risultato della ricerca.

In questo modo abbiamo abilitato Eclipse a trattare progetti Groovy.

Apriamo un nuovo progetto Groovy con menu **NEW ▸ OTHER ▸ GROOVY ▸ GROOVY PROJECT**.

Sempre con l'obiettivo di produrre semplicemente uno script del tipo di quelli visti in questo manuale, cliccando destro sulla voce `src` nella zona di sinistra dell'IDE, dove abbiamo la struttura del progetto, scegliamo **NEW ▸ FILE**, indicando poi il nome da dare allo script stesso con l'estensione `.groovy` (dobbiamo scrivere noi l'estensione in quanto Eclipse non la assegna automaticamente come fa Idea).

In questo modo l'editor è dedicato a Groovy e non appena abbiamo scritto le prime lettere di un comando, con la pressione contemporanea dei tasti **Ctrl** e barra spaziatrice ce ne viene suggerito il completamento o indicate le scelte da fare per il completamento.

Inoltre, una volta creato un oggetto, scrivendo il nome di quell'oggetto seguito da un punto (`.`) apriamo un menu a discesa che elenca tutti i metodi collegati a quell'oggetto, dandoci modo di scegliere quello che ci serve.

Se lavoriamo con Groovy avendo importato una libreria Java tutto ciò avviene anche per i metodi e gli oggetti della libreria Java importata.

14 Grafica con Groovy

Il linguaggio di programmazione Groovy è una semplificazione del linguaggio Java. Con esso, infatti, scriviamo, con una sintassi molto più semplice di quella richiesta da Java, programmi che girano sulla Java Virtual Machine e che hanno la stessa potenza di programmi scritti in linguaggio Java, anche se senza essere all'altezza di affrontare progetti molto impegnativi con le garanzie di affidabilità di Java.

Dal momento che tutte le librerie Java sono a disposizione di Groovy, lo sono anche le librerie grafiche.

Nel campo della grafica un primo obiettivo può essere quello di realizzare interfacce grafiche per i nostri programmi, le così dette GUI (Graphical User Interface).

In questo caso la ponderosa parte del programma per elaborazioni varie di natura non grafica è convenientemente scrivibile nel relativamente semplice linguaggio Groovy e la grafica serve solo per realizzare un interfacciamento con l'utente un tantino più simpatico della fredda riga di comando.

Per fare questo non dobbiamo affrontare eccessive difficoltà, soprattutto se ci accontentiamo di cose normali. E anche la necessaria integrazione tra linguaggio Groovy e linguaggio Java anche per cose un tantino più complicate non è poi così difficile.

Altro obiettivo può essere quello di creare grafica del tipo di quella che vivacizzava le applet Java di buona memoria, quella grafica che utilizza la libreria Java Graphics, con cui realizzare disegni e figure di varia natura.

In questo caso la parte preponderante del programma è quella della realizzazione del disegno, che va scritta in linguaggio Java, e Groovy serve solo per far vedere ciò che abbiamo programmato.

Teoricamente con questa libreria potremmo realizzare grafici di statistica descrittiva (istogrammi, torte, ecc.) ma il lavoro di programmazione sarebbe improponibile.

Purtroppo, infatti, Java non ci offre strumenti per fare queste cose in maniera semplice e servirebbero librerie esterne di arricchimento, come `jfreechart` o simili, il cui utilizzo esula dalle capacità di un dilettante.

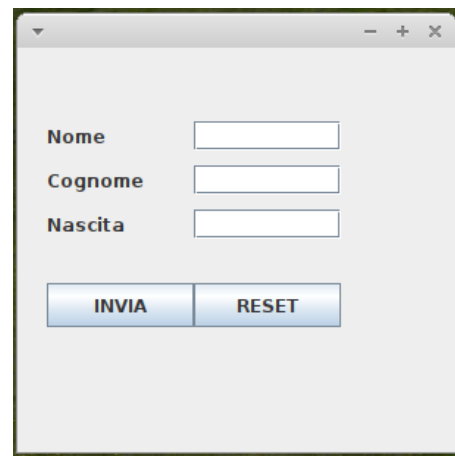
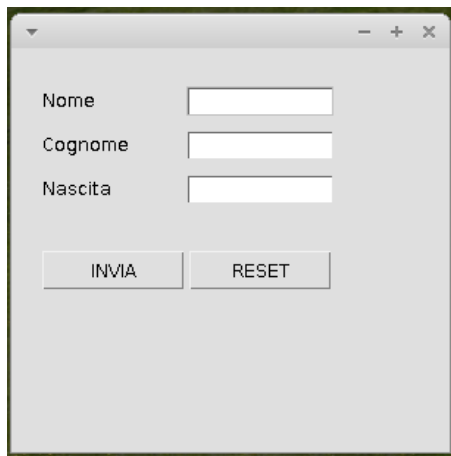
In questo manualletto vediamo semplicemente come arricchire i nostri programmi con GUI e come creare qualche giochetto grafico.

Il package Java originariamente dedicato agli strumenti per creare interfacce grafiche utente è **java.awt** (Java Abstract Widget Toolkit).

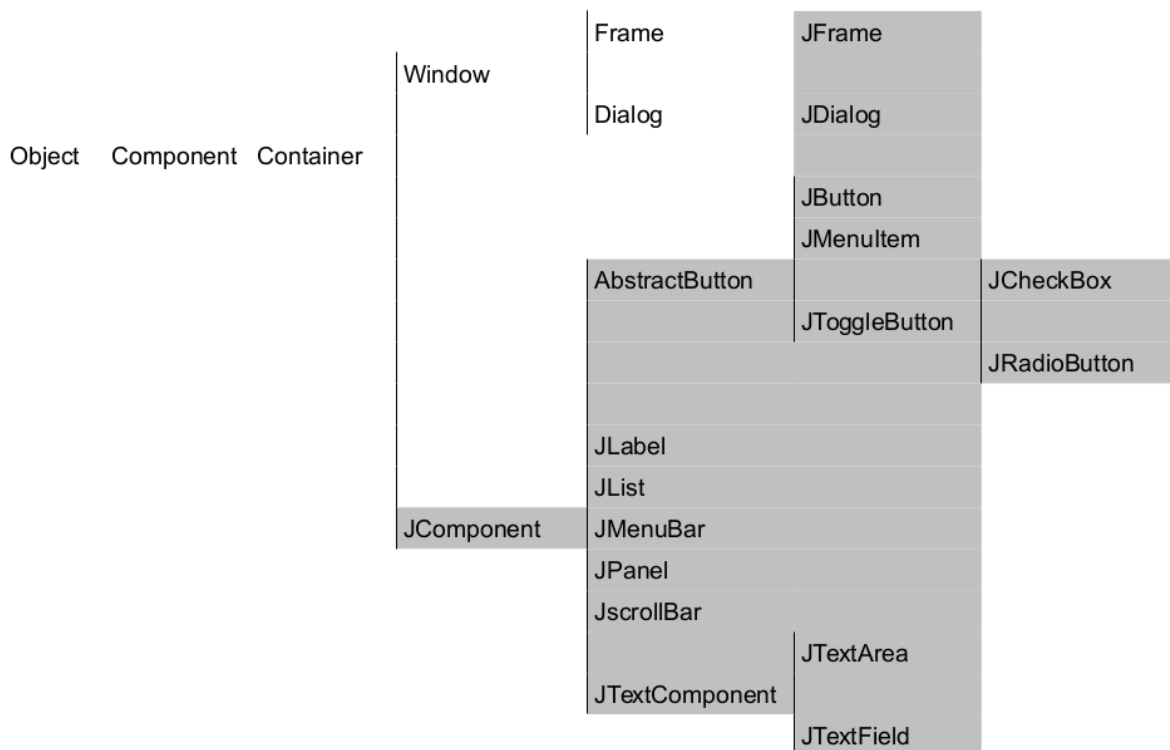
Gli strumenti grafici contenuti in questo package sono alquanto grezzi e, soprattutto, non garantiscono comportamenti uniformi su diversi sistemi operativi e su diversi hardware.

Per un sistema che si fregia del motto «write once run anywhere» non è il massimo e si è rimediato con l'apprestamento di un altro package che integra `java.awt`, che si chiama **javax.swing** e che risolve questi problemi.

Queste due illustrazioni fanno vedere la differenza di resa grafica tra un programma che utilizza `java.awt` (sulla sinistra) e un programma che utilizza `javax.swing` (sulla destra).



La gerarchia di classi che evidenzia l'integrazione tra le componenti grafiche dei due packages è la seguente:

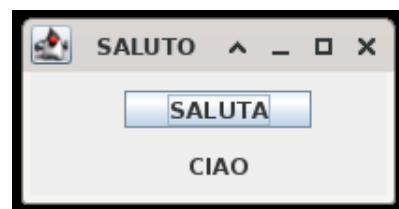
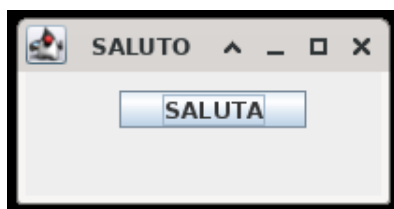


La zona ombreggiata rappresenta le classi di javax.swing che sostituiscono classi di java.awt: praticamente esse hanno lo stesso nome con una J davanti.

Il pulsante si chiama Button in java.awt e si chiama JButton in javax.swing.

Poniamo che il nostro obiettivo sia di creare una semplicissima GUI nella quale cliccando su un pulsante veniamo salutati con un Ciao.

Una cosa di questo tipo



con sulla sinistra la GUI al suo lancio e sulla destra la GUI dopo aver cliccato sul pulsante SALUTA. Utilizzando l'apparato di scripting di Groovy abbiamo tre modi di raggiungere questo obiettivo.

con linguaggio Java

L'interprete Groovy riconosce il linguaggio Java puro. Pertanto, se conosciamo Java e le sue librerie grafiche, possiamo stilare lo script in questo modo

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
class Saluto
{
    public static void main(String[] args)
    {
        JFrame mioFrame = new JFrame("SALUTO")
        mioFrame.setLayout(null)
        mioFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
        mioFrame.setBounds(200,200,200,100)
        JButton pulsante = new JButton("SALUTA")
        pulsante.setBounds(50,10,100,20)
        mioFrame.add(pulsante)
        JLabel messaggio = new JLabel()
        messaggio.setBounds(50,40,100,20)
        messaggio.setHorizontalAlignment(JLabel.CENTER)
        mioFrame.add(messaggio)
        pulsante.addActionListener(new ActionListener()
        {
            @Override
            public void actionPerformed(ActionEvent e)
            {
                messaggio.setText("CIAO");
            }
        });
        mioFrame.setVisible(true)
    }
}
```

Le indentazioni non servirebbero ma le ho usate per rendere più evidente la struttura dello script.

La cosa più ostica per un principiante è la macchinosa gestione dell'evento corrispondente alla pressione del pulsante.

Per il resto il tutto è abbastanza abbordabile, anche perché ho evitato di ricorrere alla complicata gestione del layout attraverso il layout manager di Java.

con linguaggio Groovy

Pur non potendo prescindere dalla conoscenza della libreria `javax.swing` e dei suoi meccanismi possiamo redigere lo script senza conoscere il linguaggio Java in questo modo

```

import javax.swing.*
mioFrame = new JFrame("SALUTO")
mioFrame.setLayout(null)
mioFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
mioFrame.setBounds(200,200,200,100)
JButton pulsante = new JButton("SALUTA")
pulsante.setBounds(50,10,100,20)
mioFrame.add(pulsante)
JLabel messaggio = new JLabel()
messaggio.setBounds(50,40,100,20)
messaggio.setHorizontalAlignment(JLabel.CENTER)
mioFrame.add(messaggio)
pulsante.addActionListener
{
    messaggio.setText("CIAO")
}
mioFrame.setVisible(true)

```

Qui abbiamo la gestione molto facilitata dell'evento corrispondente alla pressione del pulsante. Per il resto come prima.

con il tool Swing Builder

Fedele alla sua vocazione stenografica, Groovy ci offre un tool destinato a semplificare anche la costruzione e il maneggio dei widget di `javax.swing`.

E' il package `groovy.swing.SwingBuilder`

Utilizzando questo strumento lo script diventa il seguente.

```

import groovy.swing.SwingBuilder
import javax.swing.*
def swing = new SwingBuilder()
mioFrame = swing.frame(title:'SALUTO', location: [200,200], size:[200,100], layout: null)
{
    button(text:"SALUTA", location:[50,10], size:[100,20],actionPerformed:{messaggio.text ="CIAO"})
    messaggio = label(horizontalAlignment: JLabel.CENTER,location:[50,40], size:[100,20])
}
mioFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
mioFrame.show()

```

Per la verità, come si può vedere dal raffronto tra lo script redatto con linguaggio Groovy e questo, la semplificazione è molto relativa e ritengo che, una volta acquisite le definizioni e i metodi dei widget del package `javax.swing`, sia molto più semplice redigere lo script con il linguaggio Groovy senza bisogno di imparare altro.

Nel seguito rammento pertanto i più importanti widget del package `javax.swing` e il modo di utilizzarli con il linguaggio Groovy.

14.1 Oggetti contenitori

L'oggetto contenitore previsto da Java per le GUI è il `Frame` (in italiano Telaio): noi usiamo quello della libreria Swing, che si chiama **`JFrame`**.

`JFrame` intelaia una finestra che ha l'aspetto di quelle che abbiamo visto nella pagina precedente.

Tipicamente abbiamo la barra superiore



che contiene il titolo della finestra e i pulsanti standardizzati per il trattamento della finestra in un sistema a finestre.

La parte sottostante è predisposta per accogliere una barra per il menu dell'applicazione e l'area che vediamo è predisposta per accogliere gli oggetti, chiamati componenti (noi useremo i `JComponent` della libreria Swing), con cui costruire l'applicazione: questa area si chiama `ContentPane`.

Uno dei componenti che possiamo inserire in quest'area è il **JPanel**, che è a sua volta un contenitore.

Per costruire GUI complesse l'utilizzo dei pannelli ci dà modo di suddividere il `ContentPane` in tante sotto-aree contenitrici, a ciascuna delle quali applicare una diversa geometria per la localizzazione dei componenti attraverso quello che in Java si chiama `LayoutManager`.

La gestione di tutto questo è tutt'altro che semplice e direi che per i nostri obiettivi dilettanteschi e per evitare eccessive complicazioni possiamo rinunciare all'utilizzo dei pannelli e del `LayoutManager`.

Costruiamo il nostro telaio con:

```
new JFrame()
```

e creiamo così una finestra senza titolo.

Per dare un titolo alla finestra possiamo inserirlo tra le parentesi tonde come stringa (tra apici semplici o doppi).

Il linguaggio Java prevede per questo oggetto decine di metodi e qui cito quelli che servono per fare le cose più importanti e meno ricercate.

```
setTitle(<stringa_titolo>)
```

per dare un titolo ad una finestra creata senza titolo o per modificarlo

```
setLocation(x, y)
```

per localizzare la finestra sullo schermo

(`x` e `y` sono numeri interi indicanti le coordinate del punto in cui si colloca l'angolo in alto a sinistra della finestra)

```
setSize(<larghezza>, <altezza>)
```

per dimensionare la finestra

(`<larghezza>` e `<altezza>` sono numeri interi indicanti rispettivamente i pixel di larghezza e di altezza della finestra)

```
setBounds(x, y, <larghezza>, <altezza>)
```

per indicare, ad un tempo, localizzazione e dimensione della finestra

```
setVisible(<valore_booleano>)
```

per rendere visibile (`true`) o nascondere (`false`) la finestra

```
add(<componente>)
```

per inserire un componente nella finestra

```
setLayout(LayoutManager <tipo_layout>)
```

per eventualmente scegliere un tipo di gestione del layout.

Come ho detto prima, noi dilettanti rinunciamo al layout management e, tra le parentesi tonde, indichiamo semplicemente la parola `null`

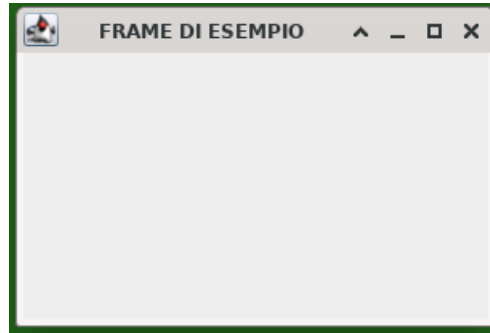
Nel caso non prevedessimo nella nostra GUI altri modi per chiudere la finestra e ci affidassimo a quello di default del sistema a finestre standard (cliccare sulla `X` in alto a destra della finestra), per chiudere anche il terminale da cui abbiamo aperta la finestra stessa lanciando lo script Groovy utilizziamo il metodo

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE).
```

Con questo script

```
import javax.swing.*
mioFrame = new JFrame("FRAME DI ESEMPIO")
mioFrame.setBounds(200, 100, 300, 200)
mioFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
mioFrame.setVisible(true)
```

otteniamo la finestra



larga 300 pixel e alta 200 pixel, posizionata in alto a sinistra dello schermo, a 100 pixel dall'alto e a 200 pixel dal lato sinistro.

Come faremo per gli altri oggetti componenti che ci serviranno, utilizziamo il costruttore (`new JFrame("FRAME DI ESEMPIO")`) per assegnare l'oggetto ad una variabile (`mioFrame`), che eredita e viene ad essere dotata di tutti i metodi previsti dal linguaggio Java per l'oggetto assegnato alla variabile, metodi che si richiamano utilizzando la sintassi della programmazione per oggetti `<nome_variabile>.<metodo>`

14.2 Colori

Se non si sceglie diversamente, come avviene per default in tutti i sistemi a finestre prodotti con i vari strumenti disponibili, ciò che facciamo viene reso con un unico colore di base e sue varianti: per esempio in nero le scritte e le linee, in grigio e relative sfumature le aree dei vari componenti.

Grazie al package `Java.awt` possiamo disporre di strumenti per scegliere colori diversi con cui abbellire e vivacizzare le nostre GUI.

Innanzitutto abbiamo i metodi di cui è dotato qualsiasi oggetto suscettibile di avere un colore:

```
setBackground(<colore>)
```

per indicare un colore per lo sfondo di un oggetto,

```
setForeground(<colore>)
```

per indicare un colore per le scritte contenute in un oggetto.

L'indicazione dell'argomento `<colore>` per questi metodi viene effettuata ricorrendo alla classe `Color` di `Java.awt` che può essere utilizzata come modificatore di colore o come costruttore di colore.

La sintassi del modificatore è

```
Color.<nome_colore>
```

dove `<nome_colore>` può essere `white`, `black`, `darkGray`, `gray`, `lightGray`, `blue`, `cyan`, `green`, `orange`, `magenta`, `yellow`.

La sintassi del costruttore è

```
new Color(r, g, b)
```

dove `r`, `g` e `b` sono numeri interi, compresi tra 0 e 255 indicanti, rispettivamente, l'intensità delle componenti di rosso, di verde e di blu nel colore desiderato.

Ricordo che `new Color(0,0,0)` indica nessun colore, cioè il colore nero e `new Color(255,255,255)` indica tutti i colori insieme, cioè il colore bianco.

Vediamo come possiamo colorare di verde la finestra che abbiamo costruito con lo script del precedente Capitolo.

Innanzitutto, oltre alla libreria `javax.swing`, dobbiamo importare la libreria `java.awt` con

```
import java.awt.*
```

Poi la prima idea che viene è di inserire, tra la prima e l'ultima istruzione dello script, l'istruzione `mioFrame.setBackground(Color.green)`

Eseguendo lo script ci aspettiamo di vedere la nostra finestra colorata di verde e se compilassimo lo script esso si compilerebbe senza errori.

La finestra, tuttavia, si presenta con il colore standard di prima.

La mancanza di errori di compilazione è dovuta al fatto che l'istruzione è corretta, in quanto l'oggetto `JFrame` possiede il metodo `setBackground`.

Il fatto è che con questo metodo, applicato al frame, coloriamo lo sfondo del frame stesso quando l'ultima istruzione dello script (`setVisible(true)`) in realtà rende visibile il `ContentPane` del frame.

E' pertanto su questo che dobbiamo agire e lo facciamo con l'istruzione

```
mioFrame.getContentPane().setBackground(Color.green)
```

In questo modo otteniamo la finestra



Per quanto detto, arriveremmo allo stesso risultato con l'istruzione

```
mioFrame.getContentPane().setBackground(new Color(0,255,0))
```

in cui esprimiamo il colore con il costruttore anziché con il modificatore.

Ovviamente il vantaggio del costruttore è quello di consentirci di creare qualsiasi colore, con qualsiasi sfumatura e la nostra creatività non è più limitata dal dover usare solo i colori riconosciuti dal modificatore.

14.3 Font

Per default Java scrive i caratteri utilizzando il font Tahoma, di normale aspetto e dimensionato sugli 11 punti: una cosa esteticamente accettabile.

Se la nostra creatività ha altre esigenze esiste un facile modo per cambiare questa impostazione di default ed è quello di utilizzare il costruttore

```
new Font(<stringa_nome>, Font.<serie>, <punti>)
```

dove

<stringa_nome> è il nome del font tra apici,

<serie> è l'aspetto del carattere e può essere `PLAIN`, `BOLD` o `ITALIC`,

<punti> è un numero intero che indica la dimensione del carattere.

Il font di cui indichiamo il nome deve essere uno di quelli installati sul sistema su cui lavoriamo.

Se così non fosse verrebbe ignorata l'indicazione <stringa_nome> e, fatte salve le altre due indicazioni, verrebbe utilizzato il font di default Tahoma.

Secondo la sintassi Groovy l'oggetto font così costruito lo assegniamo ad una variabile.

Qualsiasi oggetto contenente una scritta è dotato del metodo

`setFont()`

tra le cui parentesi tonde richiamiamo il nome della variabile cui abbiamo assegnato il font che vogliamo utilizzare in quell'oggetto.

14.4 Componenti indispensabili per una GUI

La GUI serve per acquisire informazioni dall'utente, per dare informazioni all'utente e per avere dall'utente istruzioni su quali compiti svolgere.

La libreria Swing ci fornisce tre strumenti atti a questi scopi: **JTextField** per il primo, **JLabel** per il secondo e, ma non solo, **JButton** per il terzo.

14.4.1 JTextField

E' una finestra che serve per immettere una sola riga di testo o un solo numero.

La costruiamo con

`new JTextField()`

Per default viene costruita con sfondo bianco e si può cambiare questo colore con il metodo `setBackground()` secondo quanto indicato nel Capitolo 3.

Per default si allinea a sinistra il testo inserito e si può cambiare questo con il metodo `setHorizontalAlignment(JTextField.<tipo_allineamento>)` dove <tipo_allineamento> può essere LEFT, CENTER o RIGHT.

Il metodo

`getText()`

legge il contenuto della finestra e lo ritorna come stringa.

14.4.2 JLabel

E' una finestra per esporre testo o numeri.

La costruiamo con

`new JLabel()`

All'interno delle parentesi possiamo indicare una stringa di testo da esporre già al momento della costruzione.

Per default acquisisce il colore di sfondo del contentPane e usa il colore nero per le scritte. Si può scegliere un colore diverso per le scritte con il metodo `setForeground()` secondo quanto indicato nel Capitolo 3.

Quanto al carattere tipografico della scritta vale quanto detto nel precedente Capitolo 4.

Il metodo

`setText(<stringa>)`

inserisce testo nella label, sostituendo il testo che eventualmente c'era prima.

Per default si allinea a sinistra il testo inserito e si può cambiare questo con il metodo `setHorizontalAlignment(JLabel.<tipo_allineamento>)` dove <tipo_allineamento> può essere LEFT, CENTER o RIGHT.

14.4.3 JButton

E' un pulsante sul quale si clicca con il mouse per far fare qualche cosa al computer.

Lo costruiamo con

`new JButton()`

All'interno delle parentesi possiamo indicare una stringa di testo da esporre sul pulsante già al momento della costruzione.

Allo stesso modo possiamo anche indicare l'indirizzo di una icona da esporre sul pulsante.

Con i metodi `setText()` e `setIcon()` possiamo inserire dopo la costruzione una scritta o una icona o modificare quelle esistenti.

Con i metodi `setBackground()` e `setForeground()` possiamo modificare i colori di default secondo quanto indicato nel Capitolo 3 e per quanto riguarda l'eventuale scelta di un carattere per la scritta diverso da quello di default vale quanto detto nel precedente Capitolo 4.

14.5 Disegno della GUI

Java ha vari strumenti per gestire il collocamento dei componenti nel content pane del Frame o nei pannelli, i layout manager.

Chi conosce il linguaggio Java sa dell'esistenza della classe `FlowLayout`, della classe `GridLayout`, della classe `BoxLayout`, della classe `BorderLayout`, ecc. e della possibilità di combinare variamente questi gestori attraverso l'utilizzo di vari `JPanel` inseriti in `JFrame` per costruire GUI anche molto complesse.

E se ci mettiamo su questa strada siamo costretti a ricercare queste combinazioni in quanto, nel realizzare la GUI, ci accorgiamo che in una certa zona della GUI stessa viene bene usare il `FlowLayout` ma in un'altra conviene usare il `GridLayout` e così via.

Se utilizziamo questi strumenti per costruire una GUI semplice ci costringiamo a complicare inutilmente il percorso per fare una cosa facile.

In questo manuale propongo di non utilizzare i layout manager ma di costruire noi il layout con le istruzioni di base che vedremo, previste dal linguaggio Java appunto per chi non voglia utilizzare i layout manager.

Ovviamente esiste una differenza tra i due percorsi e tra i risultati cui si perviene attraverso i due percorsi.

La collocazione degli elementi con i layout manager è una collocazione relativa e, per default, è relativa anche la dimensione dei vari componenti: la posizione di un componente è di fianco, sotto o sopra un altro componente e lo spazio che occupa si dimensiona in relazione allo spazio occupato dai componenti circostanti.

Se non usiamo i layout manager la collocazione e il dimensionamento dei vari componenti è assoluta: dobbiamo cioè indicare noi esattamente la dimensione del componente e il luogo dove va collocato.

La più vistosa differenza di risultato è che una GUI costruita con i layout manager sicuramente si adatta a qualsiasi tipo di schermo di computer e, se la ridimensioniamo utilizzando le maniglie laterali per allargarla, allungarla o restringerla il tracciato video della GUI, cioè il suo aspetto, il suo layout, rimarrà sempre quello e i componenti della GUI adatteranno posizionamento relativo e dimensioni a quelle del modificato telaio della GUI stessa.

Una GUI costruita con il posizionamento assoluto, senza layout manager, se la ridimensioniamo utilizzando le maniglie laterali per allargarla conserverà la stessa posizione e dimensione dei componenti che aveva prima dell'allargamento e se utilizziamo le maniglie laterali per restringerla faremo scomparire i componenti presenti nelle aree tagliate.

Fatta questa premessa, vediamo come gestire il così detto absolute layout, cioè come disegnare la GUI senza l'ausilio di alcun layout manager.

La prima cosa da fare è dichiarare questa intenzione nello script, subito dopo la creazione del frame, con l'istruzione

```
<nome_frame>.setLayout(null)
```

Dopo di che disegniamo i vari componenti avvalendoci dei seguenti metodi che ciascun componente possiede e poi li collochiamo utilizzando il metodo `.add` del Frame:

```
setLocation(x, y)
```

per indicare il punto del frame in cui collocare l'angolo in alto a sinistra del componente

(x e y sono i pixel di coordinata, essendo 0 e 0 l'origine in alto a sinistra del frame)
`setSize(<larghezza>, <altezza>)`
per indicare la dimensione del componente
(<larghezza> e <altezza> indicano larghezza e altezza del componente in pixel)
`setBounds(x, y, <larghezza>, <altezza>)`
per indicare, insieme, le due cose di prima.

14.6 Eventi

Quando si esegue un programma che si interfaccia con l'utente attraverso il terminale il computer segue il percorso previsto dal programma (siamo nella così detta programmazione imperativa), coinvolge l'utente quando previsto per fare ciò che il programma prevede e il programma termina come e quando previsto al suo interno.

Nei programmi dotati di GUI l'utente è più protagonista, può avere la possibilità di introdurre un dato prima dell'altro, può dare il via all'esecuzione del programma dopo aver controllato di avere inserito i dati giusti, può avere a disposizione diverse opzioni di esecuzione di certe parti del programma, può uscire dal programma ancor prima di eseguirlo completamente, ecc.

Per i piccoli programmi che facciamo noi dilettanti, comunque per programmi semplicemente destinati al calcolo o all'esecuzione di compiti molto specifici privi di opzionalità, la differenza è relativa: anzi, molte volte è assolutamente inutile e superfluo ricorrere a interfacce grafiche per l'utente.

Se però pensiamo, per esempio, ad un programma di word processing dobbiamo ammettere che esso non potrebbe esistere senza una GUI e l'alternativa di produrre un file di testo da terminale, anche se ancora possibile sul sistema Linux, fa ormai parte della preistoria del computer.

Quando si utilizza una GUI, oltre che costruirla con gli strumenti che abbiamo visto nei capitoli precedenti, dobbiamo prevedere il meccanismo con il quale captare i segnali che l'utente invia al computer attraverso l'interfaccia grafica, con il tasto Invio oppure con il posizionamento e il click del mouse, e collegare a quei segnali l'azione del computer.

Nel gergo Java i segnali sono detti eventi e il package `java.awt` contiene una nutrita serie di classi dotate di decine di metodi per gestire una vastissima serie di eventi.

Si tratta di un armamentario molto complesso, difficile da apprendere e di uso riservato a professionisti.

In questo manuale per principianti e dilettanti propongo una grande semplificazione e mostro come possiamo gestire gli eventi semplicemente ricorrendo ad un metodo di cui Groovy ha dotato i componenti `JTextField` e `JButton` che abbiamo visto nel Capitolo 5:

```
addActionListener{}
```

Nel caso del componente `JTextField` questo metodo capta l'evento pressione del tasto INVIO della tastiera.

Nel caso del componente `JButton` questo metodo capta l'evento click con tasto sinistro del mouse.

Tra le parentesi graffe scriviamo le istruzioni per dire al computer che cosa deve fare dal momento in cui viene captato l'evento.

Possiamo scrivere queste istruzioni su più righe tra la parentesi graffa di apertura e la parentesi graffa di chiusura.

14.7 Un programma di utilità

Applico ciò che abbiamo visto alla stesura di un programmino che mostra il colore nascente dalla combinazione dei colori rosso, verde e blu e che può servire per la scelta del colore con cui arricchire le nostre creazioni.

Lo script è il seguente:

```
import java.awt.*
import javax.swing.*
finestra = new JFrame("PROVA COLORE")
finestra.setBounds(200, 200, 250, 180)
finestra.setLayout(null)
finestra.getContentPane().setBackground(Color.cyan)
scritta_1 = new JLabel("Rosso")
scritta_2 = new JLabel("Verde")
scritta_3 = new JLabel("Blu")
scritta_1.setBounds(21, 10, 60, 20)
scritta_1.setHorizontalAlignment(JLabel.CENTER)
scritta_2.setBounds(91, 10, 60, 20)
scritta_2.setHorizontalAlignment(JLabel.CENTER)
scritta_3.setBounds(161, 10, 60, 20)
scritta_3.setHorizontalAlignment(JLabel.CENTER)
finestra.add(scritta_1)
finestra.add(scritta_2)
finestra.add(scritta_3)
input_1 = new JTextField()
input_2 = new JTextField()
input_3 = new JTextField()
input_1.setBounds(21, 30, 60, 20)
input_1.setHorizontalAlignment(JTextField.RIGHT)
input_2.setBounds(91, 30, 60, 20)
input_2.setHorizontalAlignment(JTextField.RIGHT)
input_3.setBounds(161, 30, 60, 20)
input_3.setHorizontalAlignment(JTextField.RIGHT)
finestra.add(input_1)
finestra.add(input_2)
finestra.add(input_3)
pulsante_1 = new JButton("COLORE")
pulsante_2 = new JButton("ESCI")
pulsante_1.setBounds(21, 55, 90, 20)
pulsante_2.setBounds(131, 55, 90, 20)
finestra.add(pulsante_1)
finestra.add(pulsante_2)
area_colore = new JTextField()
area_colore.setBounds(21, 80, 200, 45)
finestra.add(area_colore)
pulsante_2.addActionListener
{
    System.exit(0)
}
pulsante_1.addActionListener
{
    r = (input_1.getText()) as int
    g = (input_2.getText()) as int
    b = (input_3.getText()) as int
    area_colore.setBackground(new Color(r, g, b))
}
finestra.setVisible(true)
```

Eseguendo lo script otteniamo



Sulla sinistra abbiamo la finestra vuota in attesa dei dati per agire e sulla destra abbiamo la finestra come appare dopo che abbiamo inserito i dati richiesti e premuto il pulsante con la scritta COLORE, finestra che ora mostra un campione del colore tipo legno noce chiaro che deriva dalla combinazione RGB 250, 150, 30.

Ho ritenuto di arricchire l'applicazione con la possibilità di uscire con un metodo più professionale rispetto a quello di terminare l'applicazione cliccando sulla X in alto a destra della finestra: il metodo `System.exit(0)` di Java.

Questo è un esempio di script per il quale è necessario avere una GUI: con uno script a riga di comando, infatti, non riusciremmo mai a far vedere un colore.

14.8 Menu

L'invio di segnali al computer per creare eventi può avvenire non solo utilizzando componenti della GUI, quali la finestra di testo o il pulsante, ma anche utilizzando un menu.

Gli applicativi che utilizziamo per scrivere una lettera, per creare un file MIDI con un sequencer, per disegnare la pianta di un appartamento e quant'altro, in genere hanno addirittura una barra di menu e una barra di strumenti che servono per fare le stesse cose: la prima funziona con una serie di voci svolgibili a tendina che ci mostra le possibili scelte, la seconda ci presenta le stesse possibili scelte attraverso una serie di pulsanti con icone descrittive. La scelta di ciò che ci aggrada possiamo farla cliccando su una voce di menu oppure cliccando su un pulsante.

Senza pretendere di creare anche noi GUI così sovrabbondanti di strumenti di scelta, possono capitarci comunque situazioni in cui l'uso di un menu sarebbe la scelta migliore rispetto a quanto abbiamo visto finora.

Pertanto è bene che vediamo come si costruisce un menu utilizzando le classi contenute nel package `javax.swing`.

14.8.1 JMenuBar

Tra l'istestazione di un `Frame` e la zona visibile del `ContentPane` esiste una zona invisibile fino a quando non ha un contenuto visibile destinata ad ospitare la barra del menu.

Per utilizzare questa zona dobbiamo inserirvi questa barra e lo facciamo creandola con il costruttore

```
new JMenuBar()
```

e definendola meglio con il metodo

```
setBounds(0, 0, <larghezza_frame>, 20)
```

dove, con le coordinate 0 e 0, allochiamo la barra all'origine del frame e con gli altri due parametri assegniamo alla barra la stessa larghezza del frame (ripetendola in numero intero) e un'altezza ragionevole (20 pixel).

Perché questo funzioni ricordiamo la scelta di non utilizzare layout manager con l'istruzione `<nome_frame>.setLayout(null)`

A questo punto inseriamo la barra nel frame con

```
<nome_frame>.add(<nome_barra_menu>)
```

14.8.2 JMenu

E' l'oggetto che contiene una voce del menu, quella che compare nella barra del menu e cliccando sulla quale si apre la tendina che espone le varie scelte possibili.

Il costruttore è

```
new JMenu(<stringa_voce>)
```

dove <stringa_voce> è, tra apici, la voce di menu che deve apparire nella barra del menu.

Inseriamo questo oggetto e tutti gli eventuali altri dello stesso tipo, uno per uno, nella barra del menu con

```
<nome_barra_menu>.add(<nome_menu>)
```

14.8.3 JMenuItem

E' l'oggetto che contiene la scelta proposta da una voce di menu.

Lo costruiamo con

```
new JMenuItem(<stringa_scelta>)
```

dove <stringa_scelta> è, tra apici, la sotto-voce di menu che vediamo nella tendina che si apre cliccando sulla voce di menu.

Inseriamo questo oggetto e tutti gli eventuali altri dello stesso tipo, uno per uno, nel menu con
<nome_menu>.add(<nome_menu_item>)

A questo punto associamo a ciascun menu_item l'azione che deve svolgere il computer in corrispondenza della scelta di quel menu_item con

```
<nome_menu_item>.addActionListener{}
```

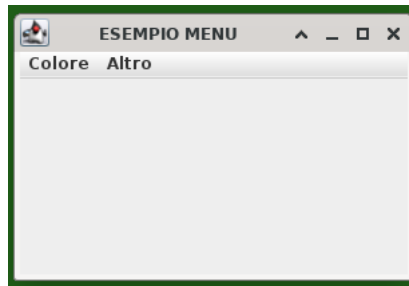
Un esempio per capirci.

Questo script

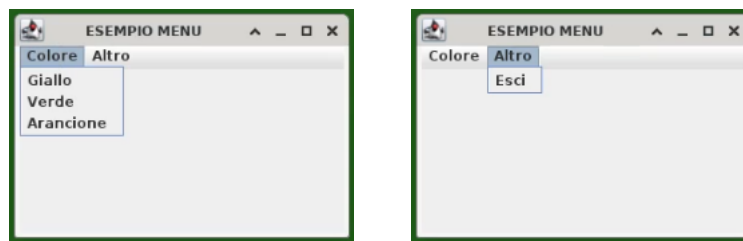
```
import javax.swing.*
import java.awt.*
mioFrame = new JFrame("ESEMPIO MENU")
mioFrame.setBounds(200,200,300,200)
mioFrame.setLayout(null)
miaMenuBar = new JMenuBar()
miaMenuBar.setBounds(0,0,300,20)
mioFrame.add(miaMenuBar)
mioMenu_1 = new JMenu("Colore")
miaMenuBar.add(mioMenu_1)
menuVoce_1 = new JMenuItem("Giallo")
mioMenu_1.add(menuVoce_1)
menuVoce_2 = new JMenuItem("Verde")
mioMenu_1.add(menuVoce_2)
menuVoce_3 = new JMenuItem("Arancione")
mioMenu_1.add(menuVoce_3)
mioMenu_2 = new JMenu("Altro")
miaMenuBar.add(mioMenu_2)
menuVoce_4 = new JMenuItem("Esci")
mioMenu_2.add(menuVoce_4)
menuVoce_1.addActionListener{mioFrame.getContentPane().setBackground(Color.yellow)}
menuVoce_2.addActionListener{mioFrame.getContentPane().setBackground(Color.green)}
menuVoce_3.addActionListener{mioFrame.getContentPane().setBackground(Color.orange)}
```

```
menuVoce_4.addActionListener{System.exit(0)}
mioFrame.setVisible(true)
```

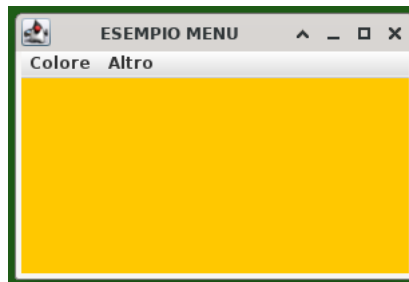
produce questa GUI



Il contenuto dei menu è il seguente



Cliccando, per esempio, su Arancione otteniamo questo



14.9 Qualche altro esempio

Potremmo essere interessati a calcolare il punto di rugiada e la temperatura percepita in presenza di una certa temperatura dell'aria e di un certo livello di umidità relativa.

La psicrometria ci fornisce le formule per fare questi calcoli.

Per determinare il punto di rugiada abbiamo la formula di Magnus-Tetens

$$punto_{rugiada} = \frac{237,7a}{17,27 - a}$$

dove, con t che esprime la temperatura e u che esprime l'umidità relativa unitaria,

$$a = \frac{17,27t}{237,7 + t} + \log(u)$$

Per determinare la temperatura percepita possiamo calcolare l'indice Humidex con la formula

$$h = t + \frac{5}{9}(e - 10)$$

dove, sempre con t che esprime la temperatura e u che esprime l'umidità relativa unitaria,

$$e = 6,11 * 10^{\frac{7,5t}{237,7 + t}} * u$$

Con questo script lo possiamo fare in maniera molto bella utilizzando le formule che abbiamo visto

```
import java.awt.*
import javax.swing.*
def double pr(double t, double u)
{
    def a = (17.27*t)/(237.7+t)+Math.log(u/100)
    return (237.7*a)/(17.27-a)
}
def double tp(double t, double u)
{
    def e = 6.11*(Math.pow(10.0,((7.5*t)/(237.7+t))))*u/100
    return t+5.0/9*(e-10)
}
finestra = new JFrame("PSICROMETRIA")
finestra.setBounds(400, 200, 300, 180)
finestra.setLayout(null)
finestra.getContentPane().setBackground(Color.green)
menu_bar = new JMenuBar()
menu_bar.setBounds(0, 0, 300, 20)
finestra.add(menu_bar)
menu = new JMenu("Calcoli")
menu_bar.add(menu)
m_i_1 = new JMenuItem("Punto di rugiada")
menu.add(m_i_1)
m_i_2 = new JMenuItem("Temperatura percepita")
menu.add(m_i_2)
frase_1 = new JLabel("Temperatura (termometro)")
frase_1.setBounds(10, 30, 250, 20)
finestra.add(frase_1)
input_1 = new JTextField()
input_1.setBounds(240, 30, 50, 20)
input_1.setHorizontalAlignment(JTextField.RIGHT)
finestra.add(input_1)
frase_2 = new JLabel("% Umidità relativa (igrometro)")
frase_2.setBounds(10, 50, 250, 20)
finestra.add(frase_2)
input_2 = new JTextField()
input_2.setBounds(240, 50, 50, 20)
input_2.setHorizontalAlignment(JTextField.RIGHT)
finestra.add(input_2)
risultato = new JLabel()
risultato.setBounds(10, 80, 280, 20)
risultato.setHorizontalAlignment(JLabel.CENTER)
finestra.add(risultato)
pulsante = new JButton("CHIUDI")
pulsante.setBounds(100, 110, 100, 20)
finestra.add(pulsante)
m_i_1.addActionListener
{
    def t = input_1.getText()
    def tt = t.toDouble()
    def u = input_2.getText()
    def uu = u.toDouble()
    def r = pr(tt, uu)
    def rr = (r*10).toInteger()
```

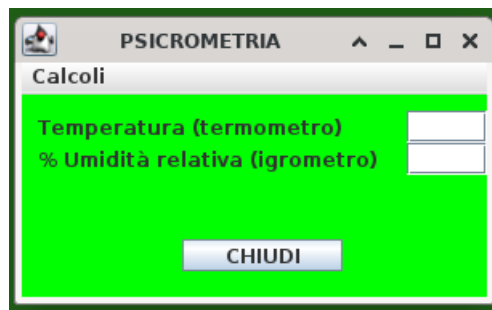


```

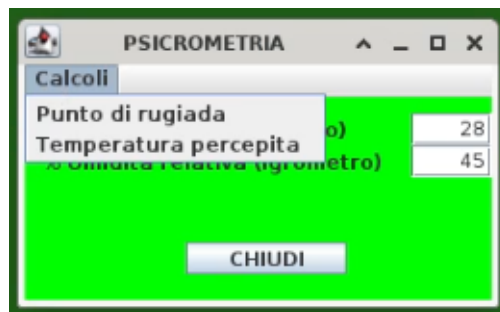
    def rrr = rr.toFloat()/10
    risultato.setText("Punto di rugiada: " + rrr.toString() + " gradi")
}
m_i_2.addActionListener
{
    def t = input_1.getText()
    def tt = t.toDouble()
    def u = input_2.getText()
    def uu = u.toDouble()
    def r = tp(tt, uu)
    def rr = (r*10).toInteger()
    def rrr = rr.toFloat()/10
    risultato.setText("Temperatura percepita: " + rrr.toString() + " gradi")
}
pulsante.addActionListener{System.exit(0)}
finestra.setVisible(true)

```

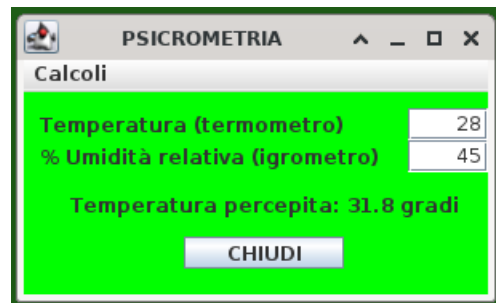
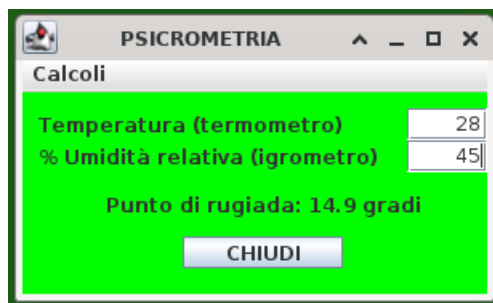
Lo script produce questa finestra



Una volta inseriti i dati della temperatura e dell'umidità relativa in percentuale, da menu scegliamo il calcolo da effettuare



Sulla sinistra abbiamo la finestra dopo aver scelto di calcolare il punto di rugiada e sulla destra abbiamo la finestra dopo aver scelto di calcolare la temperatura percepita



Un altro script per un'azione molto banale ma utile per esemplificare l'uso di colori e font.

```
import java.awt.*
import javax.swing.*
colore_1 = new Color(200, 200, 150)
colore_2 = new Color(100, 100, 150)
colore_3 = new Color(255, 30, 80)
colore_4 = new Color(160, 190, 80)
colore_5 = new Color(30, 30, 255)
font_1 = new Font("eufm10", Font.PLAIN, 22)
font_2 = new Font("MuseJazz text", Font.BOLD, 40)
finestra = new JFrame("SALUTO")
finestra.setLayout(null)
finestra.setBounds(300, 200, 300, 190)
finestra.getContentPane().setBackground(colore_1)
scritta = new JLabel("Come ti chiami?")
scritta.setForeground(colore_2)
scritta.setFont(font_1)
scritta.setBounds(10, 20, 180, 25)
finestra.add(scritta)
input = new JTextField()
input.setBounds(190, 15, 100, 25)
finestra.add(input)
saluto = new JLabel()
saluto.setBounds(10, 60, 280, 40)
saluto.setForeground(colore_3)
saluto.setFont(font_2)
saluto.setHorizontalAlignment(JLabel.CENTER)
finestra.add(saluto)
input.addActionListener
{
    nome = input.getText()
    saluto.setText("Ciao, $nome")
}
pulsante = new JButton("ESCI")
pulsante.setBounds(110, 110, 80, 30)
pulsante.setBackground(colore_4)
pulsante.setForeground(colore_5)
finestra.add(pulsante)
pulsante.addActionListener{System.exit(0)}
finestra.setVisible(true)
```

Sulla sinistra la finestra appena lanciato lo script, sulla destra la finestra dopo avere inserito il nome della persona da salutare e premuto INVIO.



14.10 Contesto grafico Java in Groovy per il disegno

Il package Java in cui troviamo gli strumenti per il disegno è **java.awt** (Java Abstract Widget Toolkit).

Questi strumenti sono metodi della classe **Graphics** e della sua sottoclasse **Graphics2D**: gli strumenti contenuti nella prima sono semplici e sufficienti per componenti di base, mentre gli strumenti contenuti nella seconda comprendono quelli della prima e altri più avanzati.

Per la realizzazione del disegno abbiamo a disposizione i metodi `paint()` della libreria `java.awt` e `paintComponent()` della libreria `javax.swing`, ai quali il sistema passa automaticamente un oggetto, chiamato `g`, della classe `Graphics` e il disegno si realizza chiamando i metodi della classe `Graphics` sull'oggetto `g`.

Se intendiamo utilizzare i metodi più evoluti della classe `Graphics2D` possiamo creare un altro oggetto, che chiamiamo `g2`, di classe `Graphics2D` attraverso conversione esplicita (casting) dell'oggetto `g`, con l'istruzione

```
Graphics2D g2 = (Graphics2D)g;
```

e realizzare il disegno chiamando i metodi della classe `Graphics2D` sull'oggetto `g2`.

Abbiamo poi il problema di dove collocare il nostro disegno, che si colloca creando una classe entro la quale richiamare i metodi `paint()` o `paintComponent()`.

Le classi disponibili, estendendo le quali creare questa classe, sono la classe **Canvas** di `java.awt` o la classe **JPanel** di `javax.swing`.

Per mostrare, infine, il nostro disegno dobbiamo istanziare l'oggetto della classe `Canvas` o `JPanel` in un `JFrame`.

Il linguaggio Groovy interviene solo in questo momento e tutto quanto abbiamo fatto prima è in puro linguaggio Java (con la sola licenza di non usare i punti e virgola a fine riga).

Pertanto le semplificazioni di Groovy in questo campo sono molto relative.

14.10.1 Strumenti per disegnare

Richiamo qui i principali metodi che ci mette a disposizione la classe `Graphics2D` per disegnare, rimandando alla documentazione Java per approfondimenti.

Metodi di disegno comuni

```
drawLine(x1, y1, x2, y2)
```

disegna una linea retta tra i punti di coordinate $(x1, y1)$ e $(x2, y2)$

```
drawRect(x, y, larghezza, altezza)
```

disegna il contorno di un rettangolo con le dimensioni indicate e l'angolo in alto a sinistra nel punto (x, y)

se larghezza e altezza sono uguali si disegna un quadrato

```
fillRect(x, y, larghezza, altezza)
```

disegna un rettangolo pieno

```
drawOval(x, y, larghezza, altezza)
```

disegna il contorno di un'ellisse iscritta in un rettangolo delle dimensioni indicate avente l'angolo in alto a sinistra nel punto (x,y)
se larghezza e altezza sono uguali si disegna un cerchio
`fillOval(x, y, larghezza, altezza)`
disegna un'ellisse piena
`drawString(stringa, x, y)`
disegna una stringa di testo a partire dal punto (x,y)

Metodi di impostazione e trasformazione

`setColor(Color.<nome>)`
imposta il colore corrente per il disegno.
<nome> può essere `black`, `white`, `red`, `green`, `blue`, `cyan`, `darkGray`, `gray`, `lightGray`, `magenta`, `orange`, `pink`, `yellow`
`setColor(new Color(R,G,B))`
imposta il colore corrente per il disegno secondo la codifica RGB
`setFont(new Font(<nome>, <stile>, <grandezza>))`
imposta il font da utilizzare per disegnare testo
<nome> è una stringa che identifica il font
<stile> può essere `Font.PLAIN`, `Font.BOLD`, `Font.ITALIC`
<grandezza> è un intero che rappresenta la dimensione del font
`setStroke(new BasicStroke(<intero>))`
imposta lo spessore della linea da disegnare
`translate(x, y)`
sposta l'origine del sistema di coordinate

14.10.2 Realizzare il disegno

Da quanto detto finora abbiamo capito che il linguaggio Java ci offre diverse alternative per scrivere programmi di disegno e dobbiamo scegliere quale utilizzare.

Personalmente ritengo che i migliori risultati siano ottenibili con una certa facilità orientandoci sull'utilizzo della libreria grafica `Graphics2D` di `java.awt` su pannello e frame `javax.swing`.

Questa scelta comporta che il nostro programma di grafica abbia la seguente struttura

```
import java.awt.*
import javax.swing.*

class <nome> extends JPanel
{
    void paintComponent(Graphics g)
    {
        Graphics2D g2 = (Graphics2D)g
        <istruzioni per il disegno>
    }
}

JFrame frame = new JFrame("<titolo>")
frame.setBounds(<int>, <int>, <int>, <int>)
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
frame.add(new <nome>())
frame.setVisible(true)
```

Spaziature e rientri non sono richiesti e li ho usati solo per evidenziare meglio la struttura del programma.

Importiamo innanzi tutto i package java che ci servono (java.awt per la libreria Graphics2D e javax.swing per le classi JPanel e JFrame e per il metodo paintComponent()).

Poi dichiariamo una nuova classe chiamata <nome> che eredita tutte le proprietà e i metodi della classe JPanel.

Al suo interno chiamiamo il metodo che viene utilizzato per disegnare sul componente grafico JPanel. La parola chiave void significa che non restituisce alcun valore, e paintComponent(Graphics g) definisce il metodo stesso, che accetta un oggetto Graphics chiamato g come argomento, su cui verranno eseguiti tutti i comandi di disegno.

Facciamo il casting in modo da sostituire all'oggetto Graphics chiamato g un oggetto Graphics2D chiamato g2.

Finalmente scriviamo le istruzioni per il disegno, utilizzando i metodi dell'oggetto g2, di cui abbiamo visto i principali nel precedente Capitolo, con la sintassi g2.<metodo>

Fin qui è tutto linguaggio Java.

Con il linguaggio Groovy proseguiamo creando un JFrame con un <titolo>, sistemandolo e dimensionandolo in modo che contenga tutto ciò che abbiamo disegnato e al suo interno istanziamo la classe del disegno e rendiamo visibile il Frame.

14.11 Esempio di grafica creativa

Nella pagina successiva si trova il programma con il quale è stato disegnata questa schermata



```
import java.awt.*
import javax.swing.*
class prove extends JPanel
{
    void paintComponent(Graphics g)
    {
        Graphics2D g2 = (Graphics2D)g
        g2.setStroke(new BasicStroke(3))
        g2.drawLine(20,20,50,50)
        g2.setStroke(new BasicStroke(5))
        g2.setColor(new Color(0,250,0))
        g2.drawOval(70,20,70,70)
        g2.setColor(Color.red)
        g2.fillOval(170,20,70,70)
        g2.setColor(Color.yellow)
        g2.fillRect(270,30,110,50)
        g2.setStroke(new BasicStroke(1))
```

```

        g2.setColor(Color.blue)
        g2.drawLine(20,120,380,120)
        g2.setFont(new Font("Courier", Font.PLAIN, 28))
        g2.drawString("Disegni di esempio", 50, 150)
        g2.drawLine(20,160,380,160)
    }
}
JFrame frame = new JFrame("ESEMPIO")
frame.setBounds(200, 200, 400, 200)
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
frame.add(new prove())
frame.setVisible(true)

```

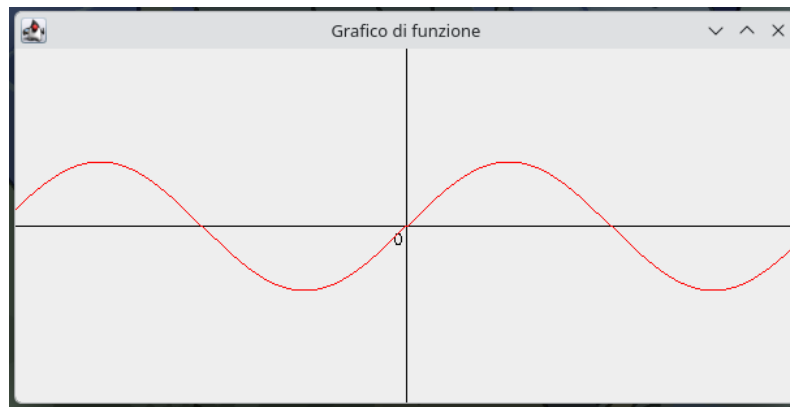
Qui è riepilogato un po' tutto ciò che abbiamo visto.

14.12 Esempio di grafica illustrativa

Utilizzando gli strumenti di disegno che abbiamo visto sarebbe possibile creare grafici di statistica descrittiva (istogrammi, torte, ecc.) ma, soprattutto se siamo in presenza di dataset di una certa consistenza possiamo immaginare a che tipo di lavoro andremmo incontro.

Purtroppo Java non ha librerie che si prestano a semplificare questo lavoro e per esigenze di questo tipo dobbiamo ricorrere a librerie esterne il cui utilizzo non è alla portata di dilettante.

Una cosa che possiamo tuttavia affrontare con relativamente meno fatica è la creazione di grafici di funzione di questo tipo



Questo è il listato del programma che ha creato il grafico

```

import java.awt.*
import javax.swing.*
class Grafico extends JPanel
{
    // La funzione da plottare: f(x) = sin(x)
    private double f(double x)
    {
        return Math.sin(x)
    }
    void paintComponent(Graphics g)
    {
        Graphics2D g2 = (Graphics2D) g

```

```

// Disegna gli assi con origine al centro
int width = getWidth()
int height = getHeight()
int originX = width/2
int originY = height/2
g2.setColor(Color.BLACK)
g2.drawLine(0,originY,width,originY) // Asse X
g2.drawLine(originX,0,originX,height) // Asse Y
g2.drawString("0",originX-10,originY+15)
// Scala: un'unità (es. 1) = 50 pixel
double scale=50.0
// Calcola e disegna i punti in rosso
g2.setColor(Color.RED)
for (int i=-originX;i<width-originX;i++)
{
    double x1=(double)i/scale
    double y1=f(x1)
    double x2=(double)(i + 1)/scale
    double y2=f(x2)
    // Converte le coordinate matematiche in coordinate pixel di Swing
    int screenX1=originX+i
    int screenY1=originY-(int)(y1*scale) // Sottrae perché l'asse Y di Swing è invertito
    int screenX2=originX+i+1
    int screenY2=originY-(int)(y2*scale)
    g2.drawLine(screenX1,screenY1,screenX2,screenY2)
}
}

JFrame frame = new JFrame("Grafico di funzione")
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
frame.add(new Grafico())
frame.setBounds(200,200,600,300)
frame.setVisible(true)

```

Ovviamente se nel costruire la funzione da plottare inseriamo la formula di un'altra funzione matematica otteniamo il grafico di quest'altra.

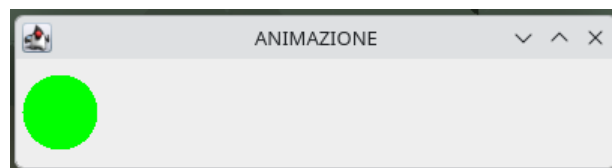
Come si vede, la scrittura del programma richiede una conoscenza più che elementare del linguaggio di programmazione Java. Purtroppo in un campo in cui a nulla servono le semplificazioni del linguaggio Groovy.

14.13 Animazioni

Dal disegno statico che abbiamo visto nel Capitolo 14, con qualche complicazione aggiuntiva possiamo passare al disegno di cose che si muovono.

Anche in questo caso la vera e propria animazione viene creata con codice Java e il linguaggio Groovy è utile solo per semplificare la fase di creazione della finestra in cui mostrare l'animazione.

Se vogliamo che in questa finestra



la pallina verde rotoli da sinistra a destra e poi scompaia scriviamo questo programma

```
import java.awt.*
import javax.swing.*
class Animazione extends JPanel
{
    // posizione iniziale
    int x = 0
    int y = 5
    // velocità orizzontale
    int dx = 2
    Animazione()
    {
        // Timer Swing: chiama muovi() e repaint() ogni 15 millisecondi
        new Timer(15,
        {
            muovi()
            repaint()
        }
        ).start()
    }
    void muovi() //modifica le coordinate di posizione dell'oggetto da disegnare
    {
        x=x+dx
    }
    void paintComponent(Graphics g)
    {
        super.paintComponent(g) //ridisegna ad ogni repaint() con la pallina nella nuova posizione
        Graphics2D g2 = (Graphics2D)g
        g2.setColor(Color.green)
        g2.fillOval(x,y,50,50)
    }
}
JFrame frame = new JFrame("ANIMAZIONE")
frame.setBounds(200,200,400,100)
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
frame.add(new Animazione())
frame.setVisible(true)
```

L'animazione viene generata dal richiamo a cadenza regolare (ottenuta dalla classe Timer di javax.swing) della funzione muovi(), che creiamo noi per modificare le coordinate dell'oggetto da muovere, e repaint(), che sta nella classe component del package java.awt, e che serve per fare ridisegnare un componente.

Parte II

Kotlin

Il linguaggio di programmazione Kotlin è stato inventato in casa JetBrains alcuni anni fa, nel 2011, per fare meglio e più facilmente tutto ciò che si può fare con il linguaggio Java.

Ricordo che JetBrains, in altri tempi chiamata IntelliJ, è la creatrice dell'ambiente di sviluppo integrato IntelliJ Idea, pensato per il linguaggio Java, disponibile liberamente con la sua edizione Community.

L'attuale ambiente di sviluppo integrato Android Studio di Google è basato su IntelliJ Idea e supporta il linguaggio Kotlin, che dal suo rilascio, nel 2012, è utilizzabile con IntelliJ Idea.

Nel tempo Google è arrivata a raccomandare Kotlin come il migliore linguaggio per programmare le app per Android, addirittura affermando che sarebbe assurdo continuare ad utilizzare Java a questo scopo, visto che c'è Kotlin.

In questo manualetto mi propongo di descrivere come Kotlin possa essere utilizzato da un programmatore alle prime armi per fare cose non eccessivamente impegnative.

Per chi si appassioni e voglia fare di più suggerisco i seguenti indirizzi:

<https://www.html.it/guide/kotlin-guida-al-linguaggio/> in italiano, sempre a livello introduttivo;

<https://kotlinlang.org/api/latest/jvm/stdlib/index.html> dove abbiamo la bibbia di Kotlin in inglese.

1 Installazione

Tra i tanti modi di lavorare con Kotlin (utilizzando il già predisposto IDE IntelliJ Idea, con plugin per altri IDE, ecc.) scelgo, per intanto, quello più semplice ed istruttivo per un principiante che deve imparare: scrivere il programma con un qualsiasi editor di testo, compilarlo ed eseguirlo da terminale (quello che in Linux e Mac OS X si chiama Terminale e in Windows si chiama Prompt dei comandi).

Un editor di testo è sicuramente presente sul nostro computer (Gedit, Kate, ecc. per chi usa Linux, Blocco Note per chi usa Windows, TextEdit per chi usa Mac).

Per l'esecuzione del programma è necessario che sul computer sia presente il JDK di Java almeno nella versione 8 e per un pieno funzionamento di Kotlin non basta il JRE che probabilmente troviamo installato con il sistema operativo.

Possiamo verificare la presenza del JDK scrivendo a terminale il comando `javac`.

Se il JDK non è installato possiamo rimediare andando all'indirizzo <https://www.oracle.com/java/technologies/download/> dove troviamo l'originale Java di Oracle. Chi usa Linux trova eventualmente la versione open source nell'installatore dei programmi.

Infine dobbiamo installare il compilatore Kotlin. La versione corrente nel momento in cui scrivo (settembre 2026) è la 2.4.10 e la troviamo all'indirizzo <https://github.com/JetBrains/kotlin/releases/tag/v2.4.10>.¹

Scorriamo la pagina e, nella zona Assets, clicchiamo sulla voce `kotlin-compiler-2.4.10.zip` per scaricare il file.

Decomprimiamo il file in una directory ospite a nostra scelta e, in questa directory, verremo ad avere la sottodirectory `kotlinc` al cui interno, nella sottodirectory `bin`, si trovano gli eseguibili per compilare e per lanciare il programma compilato.

¹Kotlin si trova anche nei pacchetti snap. Pertanto chi usa una distro Linux che supporta snap (Debian, Ubuntu, Arch, Fedora, OpenSUSE) lo può installare con il comando a terminale `sudo snap install kotlin --classic`.

Inseriamo il percorso agli eseguibili nella variabile d'ambiente PATH in modo da poterli lanciare da qualsiasi directory².

2 Come funziona: il primo programma

Java è un linguaggio nato nel 1995, nemmeno tanto lontano nel tempo, ma molto deve alla cultura di linguaggi come C e C++, rispettivamente del 1972 e del 1983, e finisce per essere meno «moderno» rispetto a un linguaggio come Python, nato quattro anni prima nel 1991.

Kotlin modernizza Java, condividendone l'ambiente operativo e facendo salvo tutto il patrimonio che nel tempo ha arricchito Java, arrivando a potersi integrare con esso utilizzandone le librerie.

Accanto all'impostazione tutta tesa alla programmazione a oggetti di Java, nato nel periodo in cui era diventato «di moda» questo modo di programmare, Kotlin, per certi versi più ancora orientato agli oggetti rispetto a Java, riscopre la programmazione funzionale e la accosta, consentendo che una funzione possa essere utilizzata non necessariamente all'interno di una classe.

Il confronto che ho proposto nell'introduzione tra i piccoli programmi di saluto mi sembra eloquente al riguardo.

Ora vediamo come si fa a programmare in Kotlin.

Per non perdere l'abitudine lanciata da Brian Kernighan negli anni settanta del secolo scorso, quando ha presentato con Dennis Ritchie il linguaggio C creato da quest'ultimo, riduciamo la prima sperimentazione del linguaggio alla creazione di un piccolissimo programma che scriva il saluto «Hello world».

Come avviene appunto nel linguaggio C, il programma parte eseguendo una funzione chiamata main, che non può mancare in nessun programma. Pertanto nel nostro editor di testo preferito scriviamo

```
fun main()
{
    println("Hello world")
}
```

Salviamo il file con un nome acconcio ed estensione .kt, per esempio, nel nostro caso, primo_programma.kt.

Ora compiliamo il nostro programma.

Apriamo il terminale, ci posizioniamo nella directory dove abbiamo salvato il file del programma e digitiamo il comando

```
kotlinc primo_programma.kt
```

seguito da INVIO.

Nella stessa directory generiamo così il file Primo_programmaKt.class che è il compilato in bytecode adatto alla Java Virtual Machine.

Lo possiamo eseguire da terminale con il comando

```
kotlin Primo_programmaKt
```

seguito da INVIO.

Sul terminale compare così la scritta Hello world.

²Ogni sistema operativo ha i suoi modi per inserire un nuovo percorso nel PATH. In Linux dobbiamo aggiungere, con potere di root, l'istruzione PATH=\$PATH:/<directory_ospite>/kotlinc/bin nel file /etc/profile. Stessa cosa si fa sul file .profile in Mac OS X. In entrambi i casi l'effetto dell'intervento si ha con il riavvio del sistema. In Windows dal Pannello di controllo si sceglie Sistema > Impostazioni di sistema avanzate > Variabili d'ambiente e si aggiunge l'indirizzo C:\<directory_ospite>\kotlinc\bin nella variabile PATH.

Se utilizziamo Snap, Kotlin si installa nella directory snap che è automaticamente aggiunta al PATH.

Nonostante, come ho detto, si tratti di un bytecode adatto alla JVM, per eseguirlo così com'è occorre il comando `kotlin` e, pertanto, occorre avere installato Kotlin sul computer: ciò in quanto, per dare in pasto il nostro programma alla JVM, abbiamo bisogno di passare attraverso una libreria di runtime che fa parte del pacchetto Kotlin.

Possiamo tuttavia compilare il nostro programma includendo questa libreria di runtime e generando un file `.jar` che potrà essere eseguito, come qualsiasi programma Java, su qualsiasi computer e con qualsiasi sistema operativo che abbia installata la JVM, anche senza avere installato Kotlin. Per generare lo `jar` eseguibile Java, che chiameremo `primo_programma.jar`, diamo il comando `kotlinc primo_programma.kt -include-runtime -d primo_programma.jar` seguito da INVIO.

Ora siamo in pieno tornati nel mondo Java. Kotlin ci è semplicemente servito per creare più facilmente un vero e proprio programma Java.

La sua esecuzione avviene con il solito comando

```
java -jar primo_programma.jar
```

seguito da INVIO.

Ma le ambizioni di Kotlin non finiscono qui.

Allo stesso indirizzo da cui abbiamo scaricato il compilatore Kotlin che stiamo usando troviamo infatti altri tre compilatori, chiamati `kotlin-native-prebuilt-linux`, `kotlin-native-prebuilt-macos` e `kotlin-native-prebuilt-windows`, utilizzando i quali possiamo compilare eseguibili rispettivamente per i sistemi operativi Linux, mac OS X e Windows, funzionanti anche senza che sul computer sia installato Java.

Francamente, visto che ormai penso non esista computer su cui non sia presente Java - e ci vuole così poco a mettercela - mi pare che ci possiamo accontentare del vecchio file `.jar` per far circolare i nostri programmi.

Senza dimenticare che, almeno per ora, non esiste la cross-compilation in Kotlin. Per cui, se siamo su Linux possiamo generare un eseguibile nativo che gira solo su Linux, se siamo su Windows possiamo generare un eseguibile nativo che gira solo su Windows, ecc. e, per la circolarità del nostro programma, le cose addirittura peggiorano rispetto alla produzione dell'archivio eseguibile `jar`.

Il tutto, poi, non ha alcun riflesso su come si scrivono i programmi, che è l'obiettivo di questo manuale.

Ultima notazione introduttiva riguarda l'inserimento dei commenti nel listato del programma. Esattamente come in Java, il commento su più righe è inserito tra i simboli `/*` e `*/` e il commento su una sola riga è preceduto dal simbolo `//`.

3 Tipi fondamentali

Fedele alla tradizione C, C++ e Java, Kotlin è un linguaggio fortemente tipizzato, a tipizzazione statica.

Nel prossimo capitolo vedremo come in Kotlin la gestione del tipo dei dati per costanti e variabili sia del tutto originale.

Per intanto immagazziniamo il concetto che Kotlin non ha, come praticamente tutti i linguaggi di programmazione, i così detti tipi primitivi (attributi con i quali si definisce il tipo di un valore) ma i tipi vengono definiti ricorrendo a classi.

Dal momento che in Java si è abituati a nominare le classi con iniziali maiuscole questo spiega come mai la definizione dei tipi in Kotlin avvenga ricorrendo a parole che cominciano con la lettera maiuscola.

Tipi numerici

Le classi per definire i tipi numerici sono

Byte per interi con una dimensione di 8 bit

Short per interi con una dimensione di 16 bit

Int per interi con una dimensione di 32 bit

Long per interi con una dimensione di 64 bit

Float per decimali con una dimensione di 32 bit

Double per decimali con una dimensione di 64 bit

Per avere un'idea delle grandezze numeriche compatibili con i vari tipi di interi occorre elevare 2 al numero dei bit.

Per i decimali ricordare che il tipo float (precisione singola) gestisce fino a 7 cifre decimali significative e il double (doppia precisione) gestisce fino a 15 cifre decimali significative.

Con i tipi numerici possiamo compiere tutte le operazioni aritmetiche:

- . somma attraverso l'operatore +
- . sottrazione attraverso l'operatore -
- . prodotto attraverso l'operatore *
- . divisione attraverso l'operatore /
- . resto intero attraverso l'operatore %

Per quanto riguarda la divisione, occorre ricordare che se sia il dividendo sia il divisore sono interi il risultato è un intero e l'eventuale parte decimale si perde.

Possiamo utilizzare numeri interi e numeri decimali con gli operatori visti sopra in espressioni finalizzate alla sola scrittura di un altro numero, come elemento letterale.

Se scriviamo `println(3.5)` otteniamo la stampa di 3.5.

Se scriviamo `println(7.0/2)` otteniamo sempre la stampa di 3.5.

Se scriviamo `println(1+2.5)` otteniamo ancora la stampa di 3.5.

Negli ultimi due casi, infatti, $7.0/2$ e $1+2.5$ erano soltanto modi diversi di scrivere il numero 3.5.

Notare come, per la divisione, almeno uno degli operandi sia stato scritto come numero decimale (7.0 e non 7) in modo da evitare la divisione tra interi ($7/2$ avrebbe dato il risultato 3).

Tipi carattere

Le classi per definire i tipi carattere sono

Char per un carattere singolo scritto tra apici semplici o doppi con dimensione di 16 bit

String per una stringa di caratteri scritta tra doppi apici

La stringa è una successione di caratteri di lunghezza fissa usata per rappresentare testo.

Il suo contenuto si crea scrivendo i caratteri tra una coppia di doppi apici " al cui interno possiamo utilizzare i caratteri di escape, come il new line `\n`.

Introducendo e chiudendo la stringa con tre doppi apici possiamo scrivere la stringa su più righe e in questo caso non vengono accettati i caratteri di escape.

Più stringhe possono essere sommate attraverso l'operatore + e il risultato è una nuova stringa contenente i caratteri delle stringhe di partenza in successione senza spazi intermedi (questi, se si vogliono nel risultato, vanno precedentemente inseriti come stringhe vuote o come caratteri vuoti alla fine delle stringhe interessate alla somma).

Possiamo contare i caratteri contenuti in una stringa utilizzando il metodo `length`, che ritorna un numero intero corrispondente al numero di caratteri contenuti nella stringa, contando anche gli eventuali spazi bianchi.

Se abbiamo una stringa `s` contenente la parola `Vittorio` con l'istruzione `println(s.length)` otteniamo la stampa del numero 8.

La posizione di ogni carattere è indicizzata partendo con l'indice 0 per il primo carattere a sinistra. Se alla stringa facciamo seguire l'indice tra parentesi quadre viene ritornato il carattere corrispondente all'indice. Sempre avendo la stringa `s` di prima, con l'istruzione `println(s[1])` otteniamo la stampa del carattere `i`, il secondo della stringa, corrispondente all'indice 1 partendo da zero.

Tipo booleano

Con la classe **Boolean** rappresentiamo i valori logici `true` e `false`.

Con i valori booleani possiamo utilizzare i tre operatori logici

`&&` che sta per `and`,

`||` che sta per `or`,

`!` che sta per `not`.

Se scriviamo `println(true && true)` otteniamo la stampa di `true`,

Se scriviamo `println(true && false)` otteniamo la stampa di `false`,

Se scriviamo `println(true || true)` otteniamo la stampa di `true`,

Se scriviamo `println(true || false)` otteniamo la stampa di `true`,

Se scriviamo `println(!true)` otteniamo la stampa di `false`.

4 Variabili e costanti

Variabili e costanti, in Kotlin, sono oggetti e abbiamo già detto, nel precedente capitolo, che per definire il tipo del dato che esse contengono abbiamo bisogno di ricorrere a classi, non ad aggettivi come avviene nei linguaggi che hanno i tipi primitivi.

Tuttavia abbiamo anche detto che il programma che scriviamo in linguaggio Kotlin gira sulla Java Virtual Machine, che riconosce i tipi primitivi del linguaggio Java. Per fare che tutto funzioni, il tipo che la classe Kotlin assegna al dato è un tipo primitivo di Java.

Se scriviamo, compiliamo ed eseguiamo il seguente programmino

```
fun main()
{
    var a: Double
    a = 32.78
    println(a.javaClass.name)
}
```

otteniamo il risultato

`double`.

La variabile `a` alla quale abbiamo assegnato il tipo con la classe `Double` (con l'iniziale maiuscola), se andiamo ad indagare di che tipo è, con l'istruzione argomento di `println()`, si rivela essere di tipo `double`, che, con la sua iniziale minuscola altro non è che un tipo primitivo di Java.

Una costante contiene un valore che non si può modificare nel corso del programma mentre una variabile contiene un valore che può essere modificato nel corso del programma.

Una costante si crea con la parola **val** mentre la variabile si crea con la parola **var**.

La sintassi canonica per creare costanti e variabili è

```
val <nome_costante>: <tipo> = <valore>
```

```
var <nome_variabile>: <tipo> = <valore>
```

Per esempio con `var nome: String = "Vittorio"` creiamo la variabile stringa identificata con nome contenente il valore `Vittorio`. Trattandosi di una variabile nulla vieta che, nel corso del programma, con l'istruzione `nome = "Maria"` il valore contenuto diventi `Maria`.

Con `val raggio: Int = 12` creiamo la costante numerica `raggio` contenente il valore 12. Trattandosi di una costante, il valore 12 che contiene non può essere modificato nel corso del programma ed ogni tentativo di farlo genera un errore.

Esiste anche la sintassi abbreviata che ci solleva dalla fatica di indicare il tipo al momento della creazione:

```
val <nome_costante> = <valore>
var <nome_variabile> = <valore>
```

Il tipo viene assegnato automaticamente da Kotlin desumendolo da ciò che scriviamo al posto di `<valore>` (ricordarsi i doppi apici per le stringhe).

Di norma il valore va indicato al momento della creazione: Kotlin normalmente usa la tecnica delle costanti e delle variabili inizializzate. Comunque pretende che nel corso del programma avvenga l'inserimento di un valore, altrimenti si generano errori di compilazione.

Per differire l'inserimento del valore, si creano costanti e variabili con

```
val <nome_costante>: <tipo>
var <nome_variabile>: <tipo>
```

sapendo che, prima o poi, sono d'obbligo le istruzioni

```
<nome_costante> = <valore>
<nome_variabile> = <valore>
```

altrimenti il programma non funziona.

Dal momento che, come ho già detto, le costanti e le variabili di Kotlin sono oggetti, esse sono dotate di vari metodi. Qui segnalo quelli che ritengo più utili.

metodi delle costanti e delle variabili numeriche

`toString()` trasforma in tipo `String` il contenuto della costante o della variabile

`toByte()` trasforma in tipo `Byte` il contenuto della costante o della variabile

`toInt()` trasforma in tipo `Int` il contenuto della costante o della variabile

`toFloat()` trasforma in tipo `Float` il contenuto della costante o della variabile

`toDouble()` trasforma in tipo `Double` il contenuto della costante o della variabile

`compareTo(<altra_costante_o_variabile_numerica>)` instaura un confronto tra la costante/variabile numerica con altra costante/variabile numerica restituendo il valore 0 se sono uguali, un valore positivo se la prima è maggiore della seconda e un valore negativo se la prima è inferiore alla seconda.

Se, per esempio, abbiamo la variabile di tipo intero `x` contenente il valore 9 e vogliamo estrarne la radice quadrata con la funzione `sqrt` del package `Math` che accetta solo parametri di tipo `double` possiamo scrivere `Math.sqrt(x.toDouble())` e otteniamo il valore 3.0. Se scrivessimo `Math.sqrt(x)` otterremmo errore.

Ricordare che il casting alla rovescia, da `Double` a `Int`, comporta la perdita della parte decimale senza arrotondamento e che tutto va fatto con un occhio alla logica: mi pare per esempio ovvio che il casting tra un `double` con sette cifre intere a un `Byte` non potrà mai funzionare.

Interessante notare che, dal momento che Kotlin riconosce il tipo adatto per quanto scriviamo sulla tastiera, i metodi che abbiamo visto si possono applicare direttamente a ciò che scriviamo. Per esempio se scriviamo `10.375.toInt()` otterremo 10. Se scriviamo `10.toDouble()` otteniamo 10.0, ecc.

metodi delle costanti e delle variabili stringa

`length` come abbiamo già visto ritorna il numero dei caratteri della stringa

`get(<indice>)` ritorna il carattere corrispondente all'indice nella stringa con tipo `Char`

`compareTo(<altra_costante_o_variabile_stringa>)` instaura un confronto tra la costante/variabile stringa con altra costante/variabile stringa restituendo il valore 0 se sono di lunghezza

uguale, un valore positivo se la prima è più lunga della seconda e un valore negativo se la prima è più corta della seconda.

metodi delle costanti e delle variabili char

`toInt()` ritorna il codice ASCII del carattere.

Se abbiamo una stringa `s` contenente la frase «che bello!», con `var l = s.length` inseriamo il valore intero 10 nella variabile `l` (la conta comprende anche lo spazio bianco). Con `s.get(3)` non scriviamo nulla, in quanto all'indice 3 della stringa corrisponde uno spazio bianco. Con `s.get(3).toInt()` otteniamo 32 che è il codice ASCII dello spazio bianco.

5 Operatori

Gli operatori collegano tra loro operandi di varia natura in espressioni che forniscono un risultato.

Operatori aritmetici

Li abbiamo già visti parlando dei tipi.

Mi limito a citare quelli di uso più comune per i principianti.

+ somma, si applica a valori numerici e stringhe,

- differenza, si applica a valori numerici,

* prodotto, si applica a valori numerici,

/ divisione, si applica a valori numerici,

% resto intero, si applica a valori numerici.

Ricordare sempre che la divisione tra interi restituisce un intero, ignorando la parte decimale. Per avere la parte decimale occorre che almeno uno degli operandi sia di tipo decimale (`float` o `double`).

Come si vede non c'è un semplice operatore per l'elevamento a potenza, operazione per la quale dobbiamo trovare altrove lo strumento.

Operatori di confronto

Servono per confrontare due valori e il risultato che restituiscono è un valore booleano.

Sono i seguenti.

`==` uguale,

`!=` non uguale,

`<` minore,

`<=` minore o uguale,

`>` maggiore,

`>=` maggiore o uguale.

Gli operandi assoggettati al confronto devono avere lo stesso tipo.

Operatori logici

Li abbiamo già visti parlando del tipo booleano, il risultato che forniscono è un valore booleano e sono i seguenti.

`&&` che sta per `and`,

`||` che sta per `or`,

`!` che sta per `not`.

6 Funzioni

La funzione, in altri contesti detta anche procedura o subroutine, è una sezione del programma in cui è racchiusa una serie di istruzioni da eseguire e che vengono eseguite ogni volta che la funzione è chiamata.

Nel Capitolo 2 abbiamo visto che in un programma Kotlin deve esistere almeno una funzione, la funzione `main`: è la funzione che viene automaticamente chiamata al lancio del programma.

Né basta questa sola funzione per far fare qualche cosa al programma: semplicemente per ottenere che il nostro programma scriva un salutino a terminale abbiamo dovuto chiamare un'altra funzione, la funzione `println()`.

Le istruzioni contenute nella funzione, salvo che noi stessi la programmiamo, ci sono ignote: sappiamo solo che, chiamando la funzione e inserendo gli eventuali parametri richiesti, otteniamo un certo risultato. Come di fronte a una scatola nera.

Per scrivere Hello world abbiamo chiamato la funzione `println`, le abbiamo passato il parametro (la stringa "Hello world") e ci siamo trovata la scritta a terminale, senza sapere minimamente quali istruzioni siano contenute nella funzione `println` per ottenere questo risultato.

Il linguaggio Kotlin ci mette a disposizione tantissime funzioni, raggruppate in packages, sia originarie del linguaggio sia appartenenti al sistema di sviluppo Java e nel seguito del manuale vedremo come utilizzarle.

Potremmo noi stessi scrivere funzioni e inserirle in packages in modo da averle a disposizione quando serve, ma nell'economia di questo manualetto per principianti direi che non è il caso di complicarci la vita per fare cose da professionisti.

Tuttavia, per semplificare la scrittura di un certo programma, potrebbe essere utile raggruppare alcune istruzioni all'interno del programma stesso, creando funzioni richiamabili, in modo da suddividere i compiti e da evitare di scrivere più volte le stesse cose.

La sintassi per dichiarare e scrivere una funzione è

```
fun <nome> (<nome_parametro>: <tipo_parametro, ...>): <tipo_risultato>
{
    <istruzioni>
    return <risultato>
}
```

dove <nome> è il nome che intendiamo dare alla funzione, ed è il nome che useremo per richiamarla, <nome_parametro> e <tipo_parametro> sono nome e tipo del parametro (dato) da inserire quando la richiamiamo, e ce ne possono essere più di uno, separati da virgola, <tipo_risultato> è il tipo del risultato che intendiamo ottenere. Le <istruzioni> sono quelle relative alle elaborazioni da effettuare sui parametri per ottenere il risultato.

La sintassi per chiamare una funzione ed eseguirla è

<nome_funzione> (<parametri>)

Come esercizio esemplificativo supponiamo di voler scrivere un programma che calcoli Combinazioni e Disposizioni, semplici senza ripetizione, di n numeri presi k a k .

Le formule che dobbiamo utilizzare sono

$$C(n, k) = \frac{n!}{(n-k)!k!} \text{ per le combinazioni,}$$

$$D(n, k) = \frac{n!}{(n-k)!} \text{ per le disposizioni,}$$

nelle quali ricorre spesso il calcolo del fattoriale.

Il fatto che la necessità di questo calcolo ricorra spesso e comporti una non semplice scrittura di istruzioni per effettuarlo suggerisce di isolare queste istruzioni in una funzione dedicata al calcolo del fattoriale in modo da poter richiamare queste istruzioni in blocco quando serve.

La funzione per calcolare il fattoriale può essere scritta così:

```
fun fattoriale (n: Int): Long
{
    if (n == 0)
    {
        return 1
    }
    return n * fattoriale (n - 1)
}
```

E' un bell'esempio di formula ricorsiva (funzione che richiama sé stessa) che ci dimostra come il linguaggio Kotlin supporti la ricorsività.

Per scrivere questa funzione sono ricorso all'istruzione `if` che il lettore ancora non conosce. E' stato necessario per inserire nella funzione la convenzione per cui il fattoriale di 0 è 1.

Il programma per calcolare combinazioni e disposizioni di 10 numeri presi 3 a 3 diventa il seguente:

```
fun fattoriale(n: Int): Long
{
    if (n == 0)
    {
        return 1
    }
    return n * fattoriale (n - 1)
}

fun main()
{
    var n = 10
    var k = 3
    var c = fattoriale(n)/(fattoriale(n-k) * fattoriale(k))
    var d = fattoriale(n)/fattoriale(n-k)
    println("combinazioni: $c")
    println("disposizioni: $d")
}
```

Salviamo il nostro programma nel file `calcolo_combinatorio.kt` e lo compiliamo.

All'esecuzione del programma verrà scritto sul terminale il risultato in questi termini:

```
combinazioni: 120
disposizioni: 720
```

7 Classi

Nonostante le attenzioni per chi ama la programmazione funzionale, Kotlin supporta in pieno la programmazione a oggetti.

Abbiamo visto, anzi, che in Kotlin tutto è un oggetto: persino le variabili e le costanti lo sono.

Tanti più esperti di me raggiungibili in rete possono ricordare o spiegare al lettore che cosa si intende per programmazione a oggetti.

Io provo a farlo capire con un esempio.

Supponiamo di avere spesso bisogno di determinare area e circonferenza del cerchio di cui ci è noto il raggio.

Se vogliamo evitare di scrivere tutte le volte le necessarie formule nella funzione main, anche perché non è detto che ce le ricordiamo (a volte si ha a che fare con formule che più difficilmente di quelle per calcolare la circonferenza sono rimandabili a memoria) possiamo creare delle funzioni che le contengono, come abbiamo fatto nel precedente Capitolo per calcolare il fattoriale oppure possiamo ricorrere alla programmazione a oggetti. Se seguiamo quest'altra strada scriviamo una Classe attraverso la quale creare un oggetto Cerchio, contenente funzioni (chiamate funzioni membro) attraverso le quali calcolare area e circonferenza.

Come si scrive una funzione lo abbiamo visto. Nel caso del nostro esempio la funzione per calcolare l'area del cerchio sarà scritta così:

```
fun area_cerchio(raggio: Double): Double
{
    return raggio * raggio * Math.PI
}
```

Nella funzione main, per calcolare l'area di un cerchio di raggio 3, scriveremo `area_cerchio(3.0)`³.

Se vogliamo seguire i dettami della programmazione a oggetti dobbiamo scrivere la classe Cerchio, dotarla delle funzioni membro per calcolare area e circonferenza, utilizzarla nella funzione main per costruire un oggetto cerchio del raggio desiderato e fare i nostri calcoli richiamando le funzioni membro di questo oggetto.

La sintassi per scrivere la classe è la seguente

```
class <nome_della_classe>(var <nome_parametro>: <tipo_parametro>, ...)
{
    <funzioni_membro>
}
```

Nel nostro caso scriveremo

```
class Cerchio(var raggio: Double)
{
    fun area(): Double
    {
        return raggio * raggio * Math.PI
    }
    fun circonferenza(): Double
    {
        return raggio * 2 * Math.PI
    }
}
```

Questa classe può essere utilizzata nella funzione main per calcolare area e circonferenza del cerchio in questo modo:

```
fun main()
{
    var mioCerchio = Cerchio(3.0)
    var area = mioCerchio.area()
    var circonferenza = mioCerchio.circonferenza()
    println("area: $area")
    println("circonferenza: $circonferenza")
}
```

³Notare l'inserimento del parametro 3 scritto 3.0 come double. Per come abbiamo costruito la funzione, infatti, il tipo del parametro raggio è Double. Se scrivessimo 3 genereremmo errore in quanto passeremmo un intero dove è atteso un double.

Innanzitutto abbiamo costruito l'oggetto mioCerchio come istanza della classe Cerchio con il parametro raggio uguale a 3.

Poi abbiamo calcolato area e circonferenza richiamando le funzioni membro dell'oggetto mioCerchio (con la classica sintassi della programmazione per oggetti indicando il nome dell'oggetto seguito da un punto e poi dal nome della funzione membro).

Infine abbiamo dato istruzioni per la stampa a terminale dei risultati che sono:

area: 28.274333882308138

circonferenza: 18.84955592153876

8 Packages

Il package è una raccolta di funzioni e/o di classi, una libreria.

Il linguaggio Kotlin ce ne mette a disposizione parecchi propri e tutti quelli del sistema di sviluppo Java, per fare praticamente tutto ciò che si può immaginare di programmare.

Anche le funzioni e le classi che creiamo noi potremmo inserirle in package, ma non è un lavoro da dilettanti, soprattutto quando si tratta di utilizzarli.

Ai principianti che vogliano conservare librerie prodotte da loro consiglio di tenerle memorizzate in un file di testo e copiare la parte che interessa nel file dove si scrive il programma, lavorando come fatto negli esempi dei capitoli precedenti: in questo modo creiamo pseudo-packages artigianali senza complicazioni.

I package propri di Kotlin non disponibili per default e quelli del sistema Java, se servono, occorre importarli con l'istruzione import come prima istruzione del file di programma, prima della funzione main e di quant'altro.

L'importazione di tutto ciò che contiene il package si fa con la sintassi

```
import <nome_pacchetto>.*
```

Se ci basta importare una sola classe possiamo farlo con la sintassi

```
import <nome_pacchetto>.<nome_classe>
```

Per tutto ciò che può fare un dilettante Kotlin mette a disposizione per default tutti i package propri che servono e, visto che stiamo usando il compilatore per la Java Virtual Machine, il pacchetto java.lang; per cui normalmente non dovremmo avere nulla da importare.

Infatti, per default, abbiamo a disposizione le seguenti funzioni di base con le quali possiamo fare praticamente tutto se lavoriamo a terminale senza interfacce grafiche.

println

Questa funzione scrive l'output a terminale usando una formattazione di default.

Gli elementi da scrivere, che passiamo come parametri, possono essere nomi di variabili (e ne verrà scritto il valore), letterali numerici o espressioni numeriche (e ne verrà scritto il risultato), stringhe (parole o frasi racchiuse tra doppi apici).

Più parametri si inseriscono separati dal simbolo +.

Per iniettare il valore di una variabile in una stringa si può inserire nella stringa il nome della variabile preceduto dal simbolo \$.

println inserisce automaticamente un new line e, dopo avere scritto, va a capo.

Esempio:

Supponendo di avere una variabile a contenente il valore 12.5, l'istruzione

```
println("3 moltiplicato 4 fa" + 3*4 + " e la variabile a vale" + a)
```

scrive a terminale quanto segue

```
3 moltiplicato 4 fa 12 e la variabile a vale 12.5
>>>
```

Lo stesso risultato si sarebbe ottenuto scrivendo

```
println("3 moltiplicato 4 fa" + 3*4 + " e la variabile a vale $a")
```

Se vogliamo applicare formattazioni particolari dobbiamo ricorrere alle direttive di formattazione, utilizzate come segnaposto in stringhe e, dopo la stringa, elencare i nomi delle variabili, nello stesso ordine dei segnaposti, come parametri del metodo `format` dell'oggetto stringa che abbiamo scritto.

Le direttive di formattazione si aprono con il simbolo `%` cui fanno seguito, per citare quelli di più ricorrente uso, i seguenti altri simboli:

`s` ad indicare l'inserimento di una stringa,

`d` ad indicare l'inserimento di un numero intero non particolarmente formattato,

`f` ad indicare l'inserimento di un numero decimale non particolarmente formattato,

`<n>d` con `n` numero intero, ad indicare l'inserimento di un intero allineato a destra in un campo di `n` caratteri,

`.<n>f` con `n` numero intero, ad indicare l'inserimento di un decimale con una parte decimale di `n` caratteri (l'ultimo dei quali arrotondato).

Esempio:

Supponiamo di avere tre variabili: `a` intera 78, `b` intera 3456 e `c` decimale 786,89567.

Se scriviamo l'istruzione

```
println("2 moltiplicato 3.5 fa %.2f e la variabile c vale %.3f".format(2*3.5, c))
```

otteniamo

```
2 moltiplicato 3.5 fa 7,00 e la variabile c vale 786,896
>>>
```

Notare che l'istruzione `format` è inserita con il punto che la separa dall'oggetto stringa di cui è metodo; il risultato della moltiplicazione `2 * 3,5` compare con i due decimali richiesti e il valore della variabile `c` è scritto con i tre decimali richiesti, con arrotondamento.

Se scriviamo l'istruzione

```
println("la variabile a vale%6d\nla variabile b vale%6d".format(a, b))
```

otteniamo

```
la variabile a vale    78
la variabile b vale   3456
>>>
```

con i due interi allineati a destra in campi di sei caratteri, come richiesto.

print

Fa tutto quello che fa `println` ma non inserisce il new line e non va a capo quando ha scritto. Per andare a capo occorre inserire il new line `\n` come stringa.

Non andando automaticamente a capo non è adatto per essere usato nella shell interattiva in quanto non compare subito ciò che ha scritto ma compare alla prossima istruzione con un a capo, mischiando le due scritte.

readln

Legge l'input dalla tastiera come stringa e lo colloca in una variabile o in una costante.

La sintassi è

```
var <nome_variabile> = readln()
```

```
val <nome_variabile> = readln()
```

Quando sulla tastiera si inseriscono cifre per un input numerico, è poi necessario il casting in variabili o costanti numeriche per poterne fare uso in calcoli.

Il casting si fa richiamando i relativi metodi `toByte()`, `toShort()`, `toInt()`, `toLong()`, `toFloat()` o `toDouble()` della variabile o costante di tipo stringa creata da `readln()`.

Package Math

Contiene costanti e funzioni matematiche che ci possono servire per qualsiasi calcolo possiamo immaginare. Esse si richiamano con la sintassi

`Math.<costante_o_funzione>`

Esaminiamole per raggruppamenti.

Costanti

Abbiamo le due più utilizzate costanti matematiche E (numero e, base dei logaritmi naturali), PI (pi greco).

Radici e potenze

`pow(<base>, <esponente>)` ritorna in double la base elevata all'esponente indicato, espressi in double,

`exp(x)` ritorna in double il numero e elevato a x, espresso in double,

`sqrt(x)` ritorna in double la radice quadrata di x, espresso in double,

Per radici ennesime dobbiamo ricorrere alla funzione `pow` indicando come esponente il reciproco dell'indice di radice, dato che $\sqrt[n]{x} = x^{\frac{1}{n}}$. Attenzione ad indicare nel rapporto 1/n il numero n come double.

Funzioni trigonometriche

Tutte le funzioni trigonometriche hanno l'argomento espresso in double radianti e ritornano un valore double. Le inverse hanno l'argomento in double e ritornano un valore in double radianti.

Abbiamo le funzioni `sin(x)`, `cos(x)`, `tan(x)` e le funzioni inverse `asin(x)`, `acos(x)`, `atan(x)`.

Abbiamo pure le seguenti funzioni di conversione

`toDegrees(x)` converte radianti in gradi, il tutto espresso in double,

`toRadians(x)` converte gradi in radianti, il tutto espresso in double.

Funzioni iperboliche

Tutte le funzioni iperboliche hanno l'argomento espresso in double e ritornano valori in double.

Abbiamo le funzioni `sinh(x)`, `cosh(x)`, `tanh(x)`.

Funzioni varie

Tutte queste funzioni operano su tipi double.

`abs(x)` ritorna il valore assoluto di x,

`ceil(x)` ritorna, in float64, il più piccolo intero maggiore o uguale a x,

`floor(x)` ritorna, in float64, il più grande intero minore o uguale a x,

`round(x)` ritorna il numero intero più prossimo a x,

`log(x)` ritorna il logaritmo naturale di x,

`log10(x)` ritorna il logaritmo decimale di x.

9 Interattività con l'utente

Negli esempi che abbiamo visto finora abbiamo scritto programmi che contenevano già i dati da elaborare e, una volta lanciati, fornivano il risultato. Se ci accontentassimo di questo, per calcolare le combinazioni e le disposizioni di 18 numeri presi 4 a 4, dovremmo prendere il programma che

abbiamo scritto nel Capitolo 6 per calcolare combinazioni e disposizioni di 10 numeri presi 3 a 3, ricopiarlo sostituendo al valore della variabile `n` il numero 18 e al valore della variabile `k` il numero 4, ricompilarlo e rilanciarlo.

Ma con quanto abbiamo visto nel precedente Capitolo 8 possiamo ora fare meglio e modificare il programma in modo che, una volta lanciato, chieda all'utente per quali valori vuole eseguire i calcoli in modo che il programma diventi finalizzato a calcolare combinazioni e disposizioni su numeri qualsiasi.

Forti di quanto abbiamo appreso basta che modifichiamo il programma scritto nel Capitolo 6 in questo modo

```
fun fattoriale (n: Int) :Long
{
    if (n == 0)
    {
        return 1
    }
    return n * fattoriale (n-1)
}
fun main()
{
    println("Quanti elementi utilizzi?")
    var n = readln().toInt()
    println("Con quanti elementi formi i raggruppamenti?")
    var k = readln().toInt()
    var c = fattoriale(n)/(fattoriale(n-k) * fattoriale(k))
    var d = fattoriale(n)/fattoriale(n-k)
    println("combinazioni:  $c")
    println("disposizioni:  $d")
}
```

Lanciando questo programma saremo richiesti di indicare il numero di elementi utilizzati (`n`), il numero di elementi per i raggruppamenti (`k`) e in un attimo vedremo il risultato. Purtroppo, a causa del pesante calcolo dei fattoriali implicati nelle formule, avremo risultati sballati se il numero di elementi (`n`) supera la ventina⁴.

10 Strutture di controllo

Come tutti i linguaggi di programmazione, Kotlin ci offre il modo di controllare l'esecuzione del programma attraverso istruzioni per condizionare l'esecuzione di un blocco al verificarsi di certe condizioni oppure per la ripetizione di blocchi.

if

L'istruzione `if`, chiamata istruzione di esecuzione condizionale (in inglese `if` è il nostro `se`), ci dà modo di assoggettare l'esecuzione di un blocco di istruzioni al verificarsi di una determinata condizione: se la condizione è vera viene eseguito il blocco di istruzioni, altrimenti si prosegue l'esecuzione del programma saltando il blocco stesso.

⁴Quello dello stack overflow, cioè del non riuscire a trattare numeri al di là di certe dimensioni, è un grande limite di tutti i linguaggi compilati (Pascal, C, C++, ADA) che si è trascinato anche nel linguaggio semi-compilato Java da cui deriva Kotlin. Per la verità nel pacchetto `java.math` esistono classi come `BigInteger` e `BigDecimal` che alzano i limiti. Se vogliamo essere tranquilli e poter usare grandi numeri senza alcuna complicazione è comunque meglio che ricorriamo al linguaggio Python, dove è tutto più semplice.

La sintassi è la seguente

```
if (<condizione>)  
{  
    <istruzioni>  
}
```

dove <condizione> è una qualsiasi espressione che relaziona due valori attraverso operatori di confronto: se la condizione si verifica vengono eseguite le istruzioni contenute tra le parentesi graffe, altrimenti si passa oltre.

L'istruzione if si presta anche all'esecuzione condizionale a due rami. Per ottenere questo dobbiamo abbinarla all'istruzione else con questa sintassi

```
if (<condizione>)  
{  
    <istruzioni>  
}  
else  
{  
    <istruzioni>  
}
```

In questo caso se la condizione si verifica vengono eseguite le istruzioni contenute nel primo blocco altrimenti vengono eseguite quelle contenute nel secondo blocco dopo else (altrimenti). In ogni caso proseguendo poi nell'esecuzione del resto del programma.

Possiamo infine gestire l'esecuzione condizionale a più rami abbinando a if l'istruzione else if con questa sintassi

```
if (<condizione>)  
{  
    <istruzioni>  
}  
else if (<condizione>)  
{  
    <istruzioni>  
}  
else if (<condizione>)  
{  
    <istruzioni>  
}  
else if (<condizione>)  
{  
    <istruzioni>  
}  
.....
```

e proseguire quanto vogliamo.

Il seguente programma

```
fun main()  
{  
    println("Inserisci un numero")  
    var x = readln().toInt()  
    if (x < 10)  
    {  
        println("Ci sei dentro")  
    }  
}
```

```
println("Ciao")
}
```

se inseriamo un numero minore di 10 stampa la frase «Ci sei dentro» e poi il saluto «Ciao»; se inseriamo 10 o un numero superiore stampa semplicemente il saluto «Ciao».

Quest'altro programma

```
fun main()
{
    println("Inserisci un numero")
    var x = readln().toInt()
    if (x < 10)
    {
        println("Ci sei dentro")
    }
    else
    {
        println("Sei fuori")
    }
    println("Ciao")
}
```

se inseriamo un numero minore di 10 stampa la frase «Ci sei dentro» e poi il saluto «Ciao»; se inseriamo 10 o un numero superiore stampa la frase «Sei fuori» e poi il saluto «Ciao».

Quest'altro ancora

```
fun main()
{
    println("Inserisci un numero")
    var x = readln().toInt()
    if (x < 10)
    {
        println("Ci sei dentro")
    }
    else if (x >= 10 && x <= 12)
    {
        println("Sei appena fuori")
    }
    else
    {
        println("Sei completamente fuori")
    }
    println("Ciao")
}
```

se inseriamo un numero minore di 10 stampa la frase «Ci sei dentro» e poi il saluto «Ciao»; se inseriamo 10 o 11 o 12 stampa la frase «Sei appena fuori» e poi il saluto «Ciao»; se inseriamo un numero superiore a 12 stampa la frase «Sei completamente fuori» e poi il saluto «Ciao».

when

Il costrutto `when` può semplificare ciò che possiamo fare con la serie di `else` e `else if`.

Con questo costrutto l'ultimo esempio che ho fatto può essere scritto così

```
fun main()
{
```



```

println("Inserisci un numero")
val numero = readln().toInt()
when (numero)
{
    in 0..9 -> println("Ci sei dentro")
    in 10..12 -> println("Sei appena fuori")
    else -> println("Sei completamente fuori")
}
println("Ciao")
}

```

Come si vede, si abbinano direttamente istruzioni semplici ai valori che può assumere una determinata variabile, che può anche essere una stringa, come vediamo nel seguente esempio.

```

fun main()
{
    println("Inserisci il colore del semaforo con lettera minuscola")
    val semaforo = readln()
    when (semaforo)
    {
        "rosso" -> println("Fermati")
        "verde" -> println("Passa tranquillo")
        "giallo" -> println("Rallenta perché dovrai fermarti")
        else -> println("Dovevi inserire uno dei colori del semaforo in minuscolo")
    }
}

```

In questo caso, a seconda del colore che scriviamo, abbiamo la risposta che ci indica come comportarci e se inseriamo una parola che non indica un colore del semaforo o se non usiamo l'iniziale minuscola come richiesto abbiamo l'invito a comportarci come si deve.

while

Si usa quando si vuole ripetere l'esecuzione di una istruzione o di un blocco di istruzioni fino a quando rimane vera (true) una determinata condizione booleana.

E' la tipica istruzione che consente di ripetere l'esecuzione un numero definito di volte utilizzando un contatore.

La sintassi per l'uso di questa istruzione è

```

var contatore = 1
while (contatore <= n)
{
    <istruzioni>
    contatore = contatore + 1
}

```

La variabile che ho battezzato `contatore` per richiamarne la natura, viene inizializzata con il valore 1. Ad ogni esecuzione delle istruzioni viene aumentata di 1 e, una volta raggiunto un valore `n` voluto, si ferma l'esecuzione. In genere viene nominata semplicemente `i`.

Questo piccolo programma

```

fun main()
{
    var i = 1
    while (i <= 3)
    {
        println("Ciao, Vittorio")
    }
}

```

```

        i = i + 1
    }
}

```

scrive la frase Ciao, Vittorio per tre volte.

Quest'altro

```

fun main()
{
    var i = 1
    while (i <= 6)
    {
        println(i)
        i = i + 1
    }
}

```

elenca i numeri da 1 a 6.

In entrambi gli esempi il blocco da ripetere contiene una sola istruzione oltre a quella di aumentare di 1 il contatore ma, ovviamente, può contenerne quante vogliamo.

for

Si usa quando si vuole ripetere l'esecuzione di una istruzione o di un blocco di istruzioni ogniqualvolta è vera (true) una determinata condizione booleana, verificata attraversando una qualsiasi struttura (stringa, range di valori, ecc.)

La sintassi per l'uso di questa istruzione è

```

<definizione di una struttura>
for (<variabile> in <struttura>)
{
    <istruzioni>
}

```

Se definiamo la struttura come un range di numeri interi (per esempio da 1 a 5) con l'istruzione for possiamo fare ciò che si fa con while.

Per scrivere tre volte il saluto «Ciao, Vittorio!» possiamo fare così:

```

fun main()
{
    var r = 1..3
    for (x in r)
    {
        println("Ciao, Vittorio")
    }
}

```

La struttura potrebbe essere una stringa e potremmo collegare l'esecuzione di un'istruzione o di un blocco di istruzioni alla presenza di un certo carattere o tipo di carattere contenuto nella stringa, come nei seguenti esempi.

Il seguente programma

```

fun main()
{
    var s = "Vittorio"
    for (x in s)
    {
        if (x.toString() == "t")
            println("trovato t")
    }
}

```

```
}
```

scrive due volte la frase «trovato t» avendo trovato due volte la lettera t nella stringa Vittorio.

Quest'altro programma

```
fun main()
{
    var s = "Vittorio abita a Milano"
    for (x in s)
    {
        if (x.isUpperCase())
            println(x)
    }
}
```

stampa le lettere V e M che sono le uniche lettere maiuscole contenute nella stringa.

11 Integrazione con Java

Fin qui abbiamo utilizzato strumenti (classi e funzioni) propri di Kotlin.

Ho ripetutamente detto, tuttavia, che Kotlin ha a disposizione tutte le librerie Java, cioè i packages che contengono le classi Java.

Importando una classe Java possiamo utilizzarla per costruire un oggetto in Kotlin e utilizzarne le funzioni membro per fare i nostri programmi con il linguaggio Kotlin.

Per esempio non mi risulta che Kotlin abbia una funzione per generare numeri casuali.

Dato che nel package `java.util` esiste la classe `Random` con cui possiamo costruire un oggetto generatore di numeri casuali possiamo importarla ed utilizzarla a questo fine.

Poniamo, per esempio, che vogliamo generare una serie di 3 numeri casuali compresi tra 0 e 10.

Lo possiamo fare con questo programmino

```
import java.util.Random
fun main()
{
    var generatore = Random()
    var limite = 10
    var contatore = 1
    while(contatore <= 3)
    {
        println(generatore.nextInt(limite))
        contatore = contatore + 1
    }
}
```

Con l'istruzione `import` importiamo la classe `Random` del package `java.util`, poi nella funzione `main()` la utilizziamo per costruire il generatore con la sintassi di Kotlin e, sempre con la sintassi di Kotlin, facciamo tutto il resto, salvo quando richiamiamo la funzione generatrice (`nextInt()`) che è definita nel linguaggio Java della classe `Random`.

12 Arricchire i programmi con GUI

Gli autori di Kotlin non hanno sin qui ritenuto di dotare questo linguaggio di packages di grafica.

Per vestire i nostri programmi Kotlin di interfaccia grafica (Graphical User Interface) abbiamo a disposizione questi strumenti:

- . Compose Multiplatform: framework moderno e ufficiale di JetBrains basato su Jetpack Compose. Permette di condividere l'interfaccia tra Desktop (Windows, macOS, Linux), Android e Web;
- . TornadoFX: wrapper Kotlin per JavaFX, la libreria grafica divenuta il moderno standard per l'ecosistema Java;
- . Swing: il framework GUI nativo storico di Java, direttamente utilizzabile in Kotlin senza librerie aggiuntive, semplicemente importando le librerie che servono, come abbiamo fatto nel precedente Capitolo e nel relativo esempio.

Ritengo che quest'ultima strada sia la più conveniente per un principiante e qui la illustro.

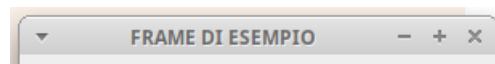
Come premessa al discorso rimando a quanto detto nella prima parte del Capitolo 14 della Parte I relativamente alle librerie `java.awt` e `java.swing`, che sono le librerie Java utilizzabili per costruire una GUI.

12.1 Oggetti contenitori

L'oggetto contenitore previsto da Java per le GUI è il `Frame` (in italiano Telaio): noi usiamo quello della libreria Swing, che si chiama **JFrame**.

`JFrame` intelaia una finestra che ha l'aspetto di quelle che abbiamo visto nella pagina precedente.

Tipicamente abbiamo la barra superiore



che contiene il titolo della finestra e i pulsanti standardizzati per il trattamento della finestra in un sistema a finestre.

La parte sottostante è predisposta per accogliere una barra per il menu dell'applicazione e l'area che vediamo è predisposta per accogliere gli oggetti, chiamati componenti (noi useremo i `JComponent` della libreria Swing), con cui costruire l'applicazione: questa area si chiama `ContentPane`.

Uno dei componenti che possiamo inserire in quest'area è il **JPanel**, che è a sua volta un contenitore.

Per costruire GUI complesse l'utilizzo dei pannelli ci dà modo di suddividere il `ContentPane` in tante sotto-aree contenitrici, a ciascuna delle quali applicare una diversa geometria per la localizzazione dei componenti attraverso quello che in Java si chiama `LayoutManager`.

La gestione di tutto questo è tutt'altro che semplice e direi che per i nostri obiettivi dilettanteschi e per evitare eccessive complicazioni possiamo rinunciare all'utilizzo dei pannelli e del `LayoutManager`.

Il metodo costruttore del nostro telaio è:

```
JFrame()
```

e crea una finestra senza titolo.

Per dare un titolo alla finestra possiamo inserirlo tra le parentesi tonde come stringa (tra doppi apici).

Il linguaggio Java prevede per questo oggetto decine di metodi e qui cito quelli che servono per fare le cose più importanti e meno ricercate.

```
setTitle(<stringa_titolo>)
```

per dare un titolo ad una finestra creata senza titolo o per modificarlo

```
setLocation(x, y)
```

per localizzare la finestra sullo schermo

(`x` e `y` sono numeri interi indicanti le coordinate del punto in cui si colloca l'angolo in alto a sinistra della finestra)

setSize(<larghezza>, <altezza>)
 per dimensionare la finestra
 (<larghezza> e <altezza> sono numeri interi indicanti rispettivamente i pixel di larghezza e di altezza della finestra)

setBounds(x, y, <larghezza>, <altezza>)
 per indicare, ad un tempo, localizzazione e dimensione della finestra

setVisible(<valore_booleano>)
 per rendere visibile (true) o nascondere (false) la finestra

add(<componente>)
 per inserire un componente nella finestra

setLayout(LayoutManager <tipo_layout>)
 per eventualmente scegliere un tipo di gestione del layout.
 Come ho detto prima, noi dilettanti rinunciamo al layout management e, tra le parentesi tonde, indichiamo semplicemente la parola null

Nel caso non prevedessimo nella nostra GUI altri modi per chiudere la finestra e ci affidassimo a quello di default del sistema a finestre standard (cliccare sulla X in alto a destra della finestra), per chiudere anche il terminale da cui abbiamo aperta la finestra stessa lanciando lo script Kotlin utilizziamo il metodo

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE).
```

Con questo script

```
import javax.swing.*
fun main()
{
  var mioFrame = JFrame("FRAME DI ESEMPIO")
  mioFrame.setBounds(200, 100, 300, 200)
  mioFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
  mioFrame.setVisible(true)
}
```

otteniamo la finestra



larga 300 pixel e alta 200 pixel, posizionata in alto a sinistra dello schermo, a 100 pixel dall'alto e a 200 pixel dal lato sinistro.

Come faremo per gli altri oggetti componenti che ci serviranno, utilizziamo il costruttore (JFrame("FRAME DI ESEMPIO")) per assegnare l'oggetto ad una variabile Kotlin (mioFrame), che eredita e viene ad essere dotata di tutti i metodi previsti dal linguaggio Java per l'oggetto assegnato alla variabile, metodi che si richiamano utilizzando la sintassi della programmazione per oggetti <nome_variabile>.<metodo>

12.2 Colori

Se non si sceglie diversamente, come avviene per default in tutti i sistemi a finestre prodotti con i vari strumenti disponibili, ciò che facciamo viene reso con un unico colore di base e sue varianti: per esempio in nero le scritte e le linee, in grigio e relative sfumature le aree dei vari componenti.

Grazie al package Java.awt possiamo disporre di strumenti per scegliere colori diversi con cui abbellire e vivacizzare le nostre GUI.

Innanzitutto abbiamo i metodi di cui è dotato qualsiasi oggetto suscettibile di avere un colore:

```
setBackground(<colore>)
```

per indicare un colore per lo sfondo di un oggetto,

```
setForeground(<colore>)
```

per indicare un colore per le scritte contenute in un oggetto.

L'indicazione dell'argomento <colore> per questi metodi viene effettuata ricorrendo alla classe Color di Java.awt che può essere utilizzata come modificatore di colore o come costruttore di colore.

La sintassi del modificatore è

```
Color.<nome_colore>
```

dove <nome_colore> può essere white, black, darkGray, gray, lightGray, blue, cyan, green, orange, magenta, yellow.

La sintassi del costruttore è

```
Color(r, g, b)
```

dove r, g e b sono numeri interi, compresi tra 0 e 255 indicanti, rispettivamente, l'intensità delle componenti di rosso, di verde e di blu nel colore desiderato.

Ricordo che Color(0,0,0) indica nessun colore, cioè il colore nero e Color(255,255,255) indica tutti i colori insieme, cioè il colore bianco.

Vediamo come possiamo colorare di verde la finestra che abbiamo costruito con lo script del precedente Capitolo.

Innanzitutto, oltre alla libreria Javax.swing, dobbiamo importare la libreria Java.awt con

```
import java.awt.*
```

Poi la prima idea che viene è di inserire, tra la prima e l'ultima istruzione dello script, l'istruzione

```
mioFrame.setBackground(Color.green)
```

Dal momento che lo script si compila senza errori, eseguendo lo script ci aspettiamo di vedere la nostra finestra colorata di verde.

Ciò, tuttavia, non avviene e la finestra si presenta con il colore standard di prima.

La mancanza di errori di compilazione è dovuta al fatto che l'istruzione è corretta, in quanto l'oggetto JFrame possiede il metodo setBackground.

Il fatto è che con questo metodo, applicato al frame, coloriamo lo sfondo del frame stesso quando l'ultima istruzione dello script (setVisible(true)) in realtà rende visibile il contentPane del frame.

E' pertanto su questo che dobbiamo agire e lo facciamo con l'istruzione

```
mioFrame.getContentPane().setBackground(Color.green)
```

In questo modo otteniamo la finestra



Per quanto detto, arriveremmo allo stesso risultato con l'istruzione `mioFrame.getContentPane().setBackground(Color(0,255,0))` in cui esprimiamo il colore con il costruttore anziché con il modificatore.

Ovviamente il vantaggio del costruttore è quello di consentirci di creare qualsiasi colore, con qualsiasi sfumatura e la nostra creatività non è più limitata dal dover usare solo i colori riconosciuti dal modificatore.

12.3 Font

Per default Java scrive i caratteri utilizzando il font Tahoma, di normale aspetto e dimensionato sugli 11 punti: una cosa esteticamente accettabile.

Se la nostra creatività ha altre esigenze esiste un facile modo per cambiare questa impostazione di default ed è quello di utilizzare il costruttore

```
Font(<stringa_nome>, Font.<serie>, <punti>)
```

dove

<stringa_nome> è il nome del font tra doppi apici,

<serie> è l'aspetto del carattere e può essere PLAIN, BOLD o ITALIC,

<punti> è un numero intero che indica la dimensione del carattere.

Il font di cui indichiamo il nome deve essere uno di quelli installati sul sistema su cui lavoriamo.

Se così non fosse verrebbe ignorata l'indicazione <stringa_nome> e, fatte salve le altre due indicazioni, verrebbe utilizzato il font di default Tahoma.

Secondo la sintassi Kotlin l'oggetto font così costruito lo assegniamo ad una variabile.

Qualsiasi oggetto contenente una scritta è dotato del metodo

```
setFont()
```

tra le cui parentesi tonde richiamiamo il nome della variabile cui abbiamo assegnato il font che vogliamo utilizzare in quell'oggetto.

12.4 Componenti indispensabili per una GUI

La GUI serve per acquisire informazioni dall'utente, per dare informazioni all'utente e per avere dall'utente istruzioni su quali compiti svolgere.

La libreria Swing ci fornisce tre strumenti atti a questi scopi: **JTextField** per il primo, **JLabel** per il secondo e, ma non solo, **JButton** per il terzo.

12.4.1 JTextField

E' una finestra che serve per immettere una sola riga di testo o un solo numero.

Il metodo costruttore è

```
JTextField()
```

Per default viene costruito con sfondo bianco e si può cambiare questo colore con il metodo `setBackground()` secondo quanto indicato nel Capitolo 3.

Per default si allinea a sinistra il testo inserito e si può cambiare questo con il metodo `setHorizontalAlignment(JTextField.<tipo_allineamento>)` dove `<tipo_allineamento>` può essere LEFT, CENTER o RIGHT.

Il metodo `getText()` legge il contenuto della finestrella e lo ritorna come stringa.

12.4.2 JLabel

E' una finestrella per esporre testo o numeri.

Il metodo costruttore è

`JLabel()`

All'interno delle parentesi possiamo indicare una stringa di testo da esporre già al momento della costruzione.

Per default acquisisce il colore di sfondo del contentPane e usa il colore nero per le scritte. Si può scegliere un colore diverso per le scritte con il metodo `setForeground()` secondo quanto indicato nel Capitolo 3.

Quanto al carattere tipografico della scritta vale quanto detto nel precedente Capitolo 4.

Il metodo

`setText(<stringa>)`

inserisce testo nella label, sostituendo il testo che eventualmente c'era prima.

Per default si allinea a sinistra il testo inserito e si può cambiare questo con il metodo `setHorizontalAlignment(JLabel.<tipo_allineamento>)` dove `<tipo_allineamento>` può essere LEFT, CENTER o RIGHT.

12.4.3 JButton

E' un pulsante sul quale si clicca con il mouse per far fare qualche cosa al computer.

Il metodo costruttore è

`JButton()`

All'interno delle parentesi possiamo indicare una stringa di testo da esporre sul pulsante già al momento della costruzione.

Allo stesso modo possiamo anche indicare l'indirizzo di una icona da esporre sul pulsante.

Con i metodi `setText()` e `setIcon()` possiamo inserire dopo la costruzione una scritta o una icona o modificare quelle esistenti.

Con i metodi `setBackground()` e `setForeground()` possiamo modificare i colori di default secondo quanto indicato nel Capitolo 3 e per quanto riguarda l'eventuale scelta di un carattere per la scritta diverso da quello di default vale quanto detto nel precedente Capitolo 4.

12.5 Disegno della GUI

Come accennato nel paragrafo 12.1 Java ha vari strumenti per gestire il collocamento dei componenti nel content pane del Frame o nei pannelli, i layout manager.

Chi conosce il linguaggio Java sa dell'esistenza della classe `FlowLayout`, della classe `GridLayout`, della classe `BoxLayout`, della classe `BorderLayout`, ecc. e della possibilità di combinare variamente questi gestori attraverso l'utilizzo di vari `JPanel` inseriti in `JFrame` per costruire GUI anche molto complesse.

E se ci mettiamo su questa strada siamo costretti a ricercare queste combinazioni in quanto, nel realizzare la GUI, ci accorgiamo che in una certa zona della GUI stessa viene bene usare il `FlowLayout` ma in un'altra conviene usare il `GridLayout` e così via.

Se utilizziamo questi strumenti per costruire una GUI semplice ci costringiamo a complicare inutilmente il percorso per fare una cosa facile.

In questo manuale propongo di non utilizzare i layout manager ma di costruire noi il layout con le istruzioni di base che vedremo, previste dal linguaggio Java appunto per chi non voglia utilizzare i layout manager.

Ovviamente esiste una differenza tra i due percorsi e tra i risultati cui si perviene attraverso i due percorsi.

La collocazione degli elementi con i layout manager è una collocazione relativa e, per default, è relativa anche la dimensione dei vari componenti: la posizione di un componente è di fianco, sotto o sopra un altro componente e lo spazio che occupa si dimensiona in relazione allo spazio occupato dai componenti circostanti.

Se non usiamo i layout manager la collocazione e il dimensionamento dei vari componenti è assoluta: dobbiamo cioè indicare noi esattamente la dimensione del componente e il luogo dove va collocato.

La più vistosa differenza di risultato è che una GUI costruita con i layout manager sicuramente si adatta a qualsiasi tipo di schermo di computer e, se la ridimensioniamo utilizzando le maniglie laterali per allargarla, allungarla o restringerla il tracciato video della GUI, cioè il suo aspetto, il suo layout, rimarrà sempre quello e i componenti della GUI adatteranno posizionamento relativo e dimensioni a quelle del modificato telaio della GUI stessa.

Una GUI costruita con il posizionamento assoluto, senza layout manager, se la ridimensioniamo utilizzando le maniglie laterali per allargarla conserverà la stessa posizione e dimensione dei componenti che aveva prima dell'allargamento e se utilizziamo le maniglie laterali per restringerla faremo scomparire i componenti presenti nelle aree tagliate.

Fatta questa premessa, vediamo come gestire il così detto absolute layout, cioè come disegnare la GUI senza l'ausilio di alcun layout manager.

La prima cosa da fare è dichiarare questa intenzione nello script, subito dopo la creazione del frame, con l'istruzione

```
<nome_frame>.setLayout(null)
```

Dopo di che disegniamo i vari componenti avvalendoci dei seguenti metodi che ciascun componente possiede e poi li collochiamo utilizzando il metodo `.add` del Frame:

```
setLocation(x, y)
```

per indicare il punto del frame in cui collocare l'angolo in alto a sinistra del componente

(x e y sono i pixel di coordinata, essendo 0 e 0 l'origine in alto a sinistra del frame)

```
setSize(<larghezza>, <altezza>)
```

per indicare la dimensione del componente

(<larghezza> e <altezza> indicano larghezza e altezza del componente in pixel)

```
setBounds(x, y, <larghezza>, <altezza>)
```

per indicare, insieme, le due cose di prima.

12.6 Eventi

Quando si esegue un programma che si interfaccia con l'utente attraverso il terminale il computer segue il percorso previsto dal programma (siamo nella così detta programmazione imperativa), coinvolge l'utente quando previsto per fare ciò che il programma prevede e il programma termina come e quando previsto al suo interno.

Nei programmi dotati di GUI l'utente è più protagonista, può avere la possibilità di introdurre un dato prima dell'altro, può dare il via all'esecuzione del programma dopo aver controllato di avere inserito i dati giusti, può avere a disposizione diverse opzioni di esecuzione di certe parti del programma, può uscire dal programma ancor prima di eseguirlo completamente, ecc.

Per i piccoli programmi che facciamo noi dilettanti, comunque per programmi semplicemente destinati al calcolo o all'esecuzione di compiti molto specifici privi di opzionalità, la differenza è relativa: anzi, molte volte è assolutamente inutile e superfluo ricorrere a interfacce grafiche per l'utente.

Se però pensiamo, per esempio, ad un programma di word processing dobbiamo ammettere che esso non potrebbe esistere senza una GUI e l'alternativa di produrre un file di testo da terminale, anche se ancora possibile sul sistema Linux, fa ormai parte della preistoria del computer.

Quando si utilizza una GUI, oltre che costruirla con gli strumenti che abbiamo visto nei capitoli precedenti, dobbiamo prevedere il meccanismo con il quale captare i segnali che l'utente invia al computer attraverso l'interfaccia grafica, con il tasto Invio oppure con il posizionamento e il click del mouse, e collegare a quei segnali l'azione del computer.

Nel gergo Java i segnali sono detti eventi e il package `java.awt` contiene una nutrita serie di classi dotate di decine di metodi per gestire una vastissima serie di eventi.

Si tratta di un armamentario molto complesso, difficile da apprendere e di uso riservato a professionisti.

In questo manuale per principianti e dilettanti propongo una grande semplificazione e mostro come possiamo gestire gli eventi semplicemente ricorrendo ad un metodo di cui sono dotati i componenti `JTextField` e `JButton` che abbiamo visto nel Capitolo 5:

```
addActionListener{}
```

Nel caso del componente `JTextField` questo metodo capta l'evento pressione del tasto INVIO della tastiera.

Nel caso del componente `JButton` questo metodo capta l'evento click con tasto sinistro del mouse.

Tra le parentesi graffe scriviamo le istruzioni per dire al computer che cosa deve fare dal momento in cui viene captato l'evento.

Possiamo scrivere queste istruzioni su più righe a patto che la prima parentesi graffa si trovi sulla stessa riga dove richiamiamo il metodo.

12.7 Un programma di utilità

Applico ciò che abbiamo visto alla stesura di un programmino che mostra il colore nascente dalla combinazione dei colori rosso, verde e blu e che può servire per la scelta del colore con cui arricchire le nostre creazioni.

Lo script è il seguente:

```
import java.awt.*
import javax.swing.*
import kotlin.system.*
fun main()
{
    var finestra = JFrame("PROVA COLORE")
    finestra.setBounds(200, 200, 220, 160)
    finestra.setLayout(null)
    finestra.getContentPane().setBackground(Color.cyan)
    var scritta_1 = JLabel("Rosso")
    var scritta_2 = JLabel("Verde")
    var scritta_3 = JLabel("Blu")
    scritta_1.setBounds(10, 10, 60, 20)
    scritta_1.setHorizontalAlignment(JLabel.CENTER)
    scritta_2.setBounds(80, 10, 60, 20)
    scritta_2.setHorizontalAlignment(JLabel.CENTER)
    scritta_3.setBounds(150, 10, 60, 20)
```

```

scritta_3.setHorizontalAlignment(JLabel.CENTER)
finestra.add(scritta_1)
finestra.add(scritta_2)
finestra.add(scritta_3)
var input_1 = JTextField()
var input_2 = JTextField()
var input_3 = JTextField()
input_1.setBounds(10, 30, 60, 20)
input_1.setHorizontalAlignment(JTextField.RIGHT)
input_2.setBounds(80, 30, 60, 20)
input_2.setHorizontalAlignment(JTextField.RIGHT)
input_3.setBounds(150, 30, 60, 20)
input_3.setHorizontalAlignment(JTextField.RIGHT)
finestra.add(input_1)
finestra.add(input_2)
finestra.add(input_3)
var pulsante_1 = JButton("COLORE")
var pulsante_2 = JButton("ESCI")
pulsante_1.setBounds(10, 55, 90, 20)
pulsante_2.setBounds(120, 55, 90, 20)
finestra.add(pulsante_1)
finestra.add(pulsante_2)
var area_colore = JTextField()
area_colore.setBounds(10, 80, 200, 45)
finestra.add(area_colore)
pulsante_2.addActionListener{exitProcess(0)}
pulsante_1.addActionListener{
    var r = (input_1.getText()).toInt()
    var g = (input_2.getText()).toInt()
    var b = (input_3.getText()).toInt()
    area_colore.setBackground(Color(r, g, b))
}
finestra.setVisible(true)
}

```

Lo salviamo con un nome che ne richiami la funzione, ad esempio prova_colore.kt.

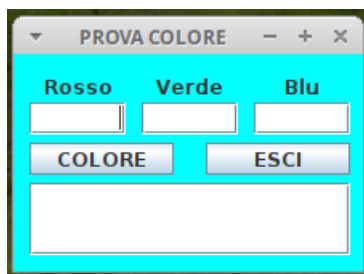
Lo compiliamo con

```
kotlinc prova_colore.kt
```

e lanciamo l'eseguibile in byte-code, che si chiama Prova_coloreKt, con

```
kotlin Prova_coloreKt
```

ottenendo



Sulla sinistra abbiamo la finestrella vuota in attesa dei dati per agire e sulla destra abbiamo la finestrella come appare dopo che abbiamo inserito i dati richiesti e premuto il pulsante con la scritta COLORE, finestrella che ora mostra un campione del colore tipo legno noce chiaro che deriva dalla combinazione RGB 250, 150, 30.

Ho ritenuto di arricchire l'applicazione con la possibilità di uscire con un metodo più professionale rispetto a quello di terminare l'applicazione cliccando sulla X in alto a destra della finestra.

Per fare questo ho dovuto utilizzare il metodo `exitProcess(0)` del package `system` di Kotlin, importato all'inizio dello script.

Se compiliamo il nostro script con

```
kotlinc prova_colore.kt -include-runtime -d prova_colore.jar
```

otteniamo il file `prova_colore.jar` che possiamo eseguire sulla Java Virtual Machine di Java anche senza avere installato sul computer Kotlin.

Questo è un esempio di programma per il quale è necessario avere una GUI: con un programma a riga di comando, infatti, non riusciremmo mai a far vedere un colore.

12.8 Menu

L'invio di segnali al computer per creare eventi può avvenire non solo utilizzando componenti della GUI, quali la finestra di testo o il pulsante, ma anche utilizzando un menu.

Gli applicativi che utilizziamo per scrivere una lettera, per creare un file MIDI con un sequencer, per disegnare la pianta di un appartamento e quant'altro, in genere hanno addirittura una barra di menu e una barra di strumenti che servono per fare le stesse cose: la prima funziona con una serie di voci svolgibili a tendina che ci mostra le possibili scelte, la seconda ci presenta le stesse possibili scelte attraverso una serie di pulsanti con icone descrittive. La scelta di ciò che ci aggrada possiamo farla cliccando su una voce di menu oppure cliccando su un pulsante.

Senza pretendere di creare anche noi GUI così sovrabbondanti di strumenti di scelta, possono capitarci comunque situazioni in cui l'uso di un menu sarebbe la scelta migliore rispetto a quanto abbiamo visto finora.

Pertanto è bene che vediamo come si costruisce un menu utilizzando le classi contenute nel package `javax.swing`.

12.8.1 JMenuBar

Tra l'intestazione di un `Frame` e la zona visibile del `ContentPane` esiste una zona invisibile fino a quando non ha un contenuto visibile destinata ad ospitare la barra del menu.

Per utilizzare questa zona dobbiamo inserirvi questa barra e lo facciamo creandola con il costruttore

```
JMenuBar()
```

e definendola meglio con il metodo

```
setBounds(0, 0, <larghezza_frame>, 20)
```

dove, con le coordinate 0 e 0, allochiamo la barra all'origine del frame e con gli altri due parametri assegniamo alla barra la stessa larghezza del frame (ripetendola in numero intero) e un'altezza ragionevole (20 pixel).

Perché questo funzioni ricordiamo la scelta di non utilizzare layout manager con l'istruzione `<nome_frame>.setLayout(null)`

A questo punto inseriamo la barra nel frame con

```
<nome_frame>.add(<nome_barra_menu>)
```

12.8.2 JMenu

E' l'oggetto che contiene una voce del menu, quella che compare nella barra del menu e cliccando sulla quale si apre la tendina che espone le varie scelte possibili.

Il costruttore è

`JMenu(<stringa_voce>)`

dove `<stringa_voce>` è, tra doppi apici, la voce di menu che deve apparire nella barra del menu.

Inseriamo questo oggetto e tutti gli eventuali altri dello stesso tipo, uno per uno, nella barra del menu con

`<nome_barra_menu>.add(<nome_menu>)`

12.8.3 JMenuItem

E' l'oggetto che contiene la scelta proposta da una voce di menu.

Il costruttore è

`JMenuItem(<stringa_scelta>)`

dove `<stringa_scelta>` è, tra doppi apici, la sotto-voce di menu che vediamo nella tendina che si apre cliccando sulla voce di menu.

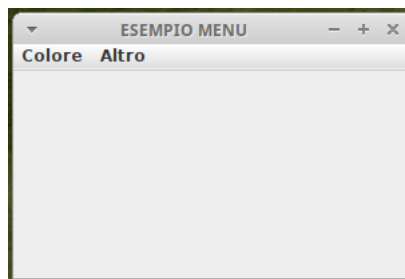
Inseriamo questo oggetto e tutti gli eventuali altri dello stesso tipo, uno per uno, nel menu con `<nome_menu>.add(<nome_menu_item>)`

A questo punto associamo a ciascun `menu_item` l'azione che deve svolgere il computer in corrispondenza della scelta di quel `menu_item` con

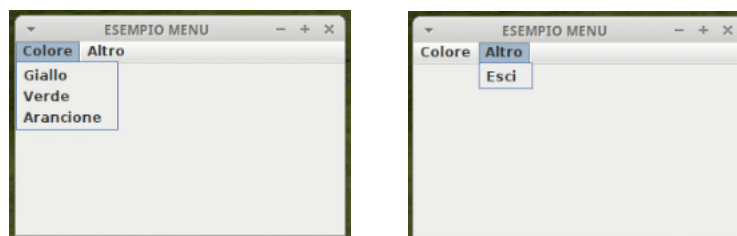
`<nome_menu_item>.addActionListener{}`

Un esempio per capirci.

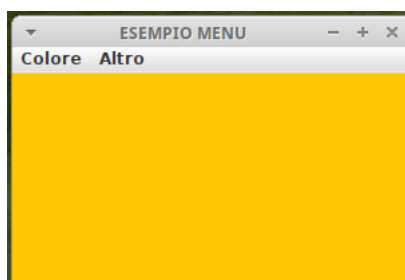
Lo script che troviamo nella pagina che segue produce questa GUI



Il contenuto dei menu è il seguente



Cliccando, per esempio, su Arancione otteniamo questo



Questo è lo script

```
import javax.swing.*
import java.awt.*
import kotlin.system.*
fun main()
{
var mioFrame = JFrame("ESEMPIO MENU")
mioFrame.setBounds(200,200,300,200)
mioFrame.setLayout(null)
var miaMenuBar = JMenuBar()
miaMenuBar.setBounds(0,0,300,20)
mioFrame.add(miaMenuBar)
var mioMenu_1 = JMenu("Colore")
miaMenuBar.add(mioMenu_1)
var menuVoce_1 = JMenuItem("Giallo")
mioMenu_1.add(menuVoce_1)
var menuVoce_2 = JMenuItem("Verde")
mioMenu_1.add(menuVoce_2)
var menuVoce_3 = JMenuItem("Arancione")
mioMenu_1.add(menuVoce_3)
var mioMenu_2 = JMenu("Altro")
miaMenuBar.add(mioMenu_2)
var menuVoce_4 = JMenuItem("Esci")
mioMenu_2.add(menuVoce_4)
menuVoce_1.addActionListener{mioFrame.getContentPane().setBackground(Color.yellow)}
menuVoce_2.addActionListener{mioFrame.getContentPane().setBackground(Color.green)}
menuVoce_3.addActionListener{mioFrame.getContentPane().setBackground(Color.orange)}
menuVoce_4.addActionListener{exitProcess(0)}
mioFrame.setVisible(true)
}
```

12.9 Qualche altro esempio

Lo script che riporto nella pagina che segue può essere utile per calcolare il punto di rugiada e la temperatura percepita in presenza di una certa temperatura dell'aria e di un certo livello di umidità relativa.

La psicrometria ci fornisce le formule per fare questi calcoli.

Per determinare il punto di rugiada abbiamo la formula di Magnus-Tetens

$$p_{\text{punto di rugiada}} = \frac{237,7a}{17,27 - a}$$

dove, con t che esprime la temperatura e u che esprime l'umidità relativa unitaria,

$$a = \frac{17,27t}{237,7 + t} + \log(u)$$

Per determinare la temperatura percepita possiamo calcolare l'indice Humidex con la formula

$$h = t + \frac{5}{9}(e - 10)$$

dove, sempre con t che esprime la temperatura e u che esprime l'umidità relativa unitaria,

$$e = 6,11 * 10^{\frac{7,5t}{237,7 + t}} * u$$

Lo script è il seguente

```
import java.awt.*
import javax.swing.*
import kotlin.system.*

fun pr(t:Double, u:Double): Double
{
    var a = (17.27*t)/(237.7+t)+Math.log(u/100)
    return (237.7*a)/(17.27-a)
}

fun tp(t:Double, u:Double): Double
{
    var e = 6.11*(Math.pow(10.0,((7.5*t)/(237.7+t))))*u/100
    return t+5.0/9*(e-10)
}

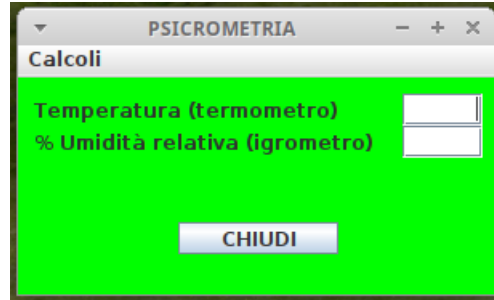
fun main()
{
    var finestra = JFrame("PSICROMETRIA")
    finestra.setBounds(400, 200, 300, 180)
    finestra.setLayout(null)
    finestra.getContentPane().setBackground(Color.green)
    var menu_bar = JMenuBar()
    menu_bar.setBounds(0, 0, 300, 20)
    finestra.add(menu_bar)
    var menu = JMenu("Calcoli")
    menu_bar.add(menu)
    var m_i_1 = JMenuItem("Punto di rugiada")
    menu.add(m_i_1)
    var m_i_2 = JMenuItem("Temperatura percepita")
    menu.add(m_i_2)
    var frase_1 = JLabel("Temperatura (termometro)")
    frase_1.setBounds(10, 30, 250, 20)
    finestra.add(frase_1)
    var input_1 = JTextField()
    input_1.setBounds(240, 30, 50, 20)
    input_1.setHorizontalAlignment(JTextField.RIGHT)
    finestra.add(input_1)
    var frase_2 = JLabel("% Umidità relativa (igrometro)")
    frase_2.setBounds(10, 50, 250, 20)
    finestra.add(frase_2)
    var input_2 = JTextField()
    input_2.setBounds(240, 50, 50, 20)
    input_2.setHorizontalAlignment(JTextField.RIGHT)
    finestra.add(input_2)
    var risultato = JLabel()
    risultato.setBounds(10, 80, 280, 20)
    risultato.setHorizontalAlignment(JLabel.CENTER)
    finestra.add(risultato)
    var pulsante = JButton("CHIUDI")
    pulsante.setBounds(100, 110, 100, 20)
    finestra.add(pulsante)
    m_i_1.addActionListener{
        var t = input_1.getText()
```

```

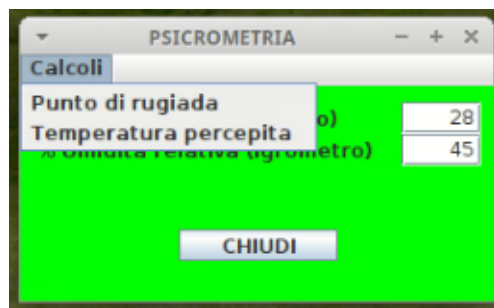
        var tt = t.toDouble()
        var u = input_2.getText()
        var uu = u.toDouble()
        var r = pr(tt, uu)
        var rr = (r*10).toInt()
        var rrr = rr.toFloat()/10
        risultato.setText("Punto di rugiada: " + rrr.toString() + " gradi")
    }
    m_i_2.addActionListener{
        var t = input_1.getText()
        var tt = t.toDouble()
        var u = input_2.getText()
        var uu = u.toDouble()
        var r = tp(tt, uu)
        var rr = (r*10).toInt()
        var rrr = rr.toFloat()/10
        risultato.setText("Temperatura percepita: " + rrr.toString() + " gradi")
    }
    pulsante.addActionListener{exitProcess(0)}
    finestra.setVisible(true)
}

```

Lo script produce questa finestra



Una volta inseriti i dati della temperatura e dell'umidità relativa in percentuale, da menu scegliamo il calcolo da effettuare



Sulla sinistra abbiamo la finestra dopo aver scelto di calcolare il punto di rugiada e sulla destra abbiamo la finestra dopo aver scelto di calcolare la temperatura percepita



Un altro script per un'azione molto banale ma utile per esemplificare l'uso di colori e font.

```
import java.awt.*
import javax.swing.*
import kotlin.system.*

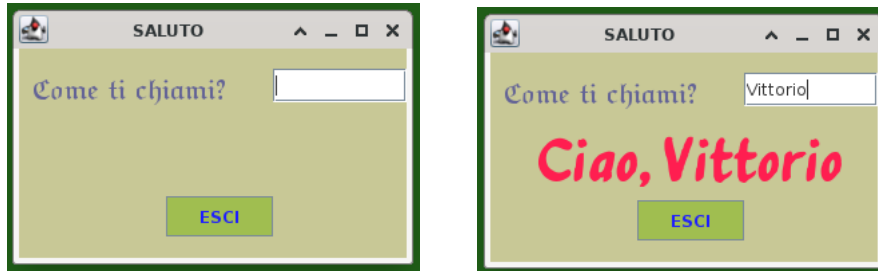
fun main()
{
    var colore_1 = Color(200, 200, 150)
    var colore_2 = Color(100, 100, 150)
    var colore_3 = Color(255, 30, 80)
    var colore_4 = Color(160, 190, 80)
    var colore_5 = Color(30, 30, 255)
    var font_1 = Font("Chilanka", Font.PLAIN, 22)
    var font_2 = Font("Learning Curve", Font.BOLD, 40)
    var finestra = JFrame("SALUTO")
    finestra.setLayout(null)
    finestra.setBounds(300, 200, 300, 190)
    finestra.getContentPane().setBackground(colore_1)
    var scritta = JLabel("Come ti chiami?")
    scritta.setForeground(colore_2)
    scritta.setFont(font_1)
    scritta.setBounds(10, 20, 180, 25)
    finestra.add(scritta)
    var input = JTextField()
    input.setBounds(190, 15, 100, 25)
    finestra.add(input)
    var saluto = JLabel()
    saluto.setBounds(10, 60, 280, 40)
    saluto.setForeground(colore_3)
    saluto.setFont(font_2)
    saluto.setHorizontalAlignment(JLabel.CENTER)
    finestra.add(saluto)
    input.addActionListener{
        var nome = input.getText()
        saluto.setText("Ciao, " + nome + "!!")
    }
    var pulsante = JButton("ESCI")
    pulsante.setBounds(110, 110, 80, 30)
    pulsante.setBackground(colore_4)
    pulsante.setForeground(colore_5)
    finestra.add(pulsante)
```

```

    pulsante.addActionListener{exitProcess(0)}
    finestra.setVisible(true)
}

```

Sulla sinistra la finestra appena lanciato lo script, sulla destra la finestra dopo avere inserito il nome della persona da salutare e premuto INVIO.



13 Utilità di un IDE

I tre classici IDE per programmare in Java sono, in ordine di anzianità, NetBeans, IntelliJ Idea e Eclipse.

Per programmare in Kotlin possiamo utilizzare gli stessi IDE sapendo tuttavia che l'utilità di Eclipse si riduce a quella di un comune editor di testo in quanto non ci fornisce la code completion per questo linguaggio.

La code completion, soprattutto se si lavora con grafica, è molto utile per alleggerire le difficoltà se non ci avvaliamo di un designer.

Il miglior IDE per programmare in Kotlin è, ovviamente, IntelliJ Idea, predisposto ad utilizzare il linguaggio Kotlin da quando questo è nato. Purtroppo la versione community non è più dotata del plugin per il designer.

NetBeans, a partire dalla versione 8.2, ci offre Kotlin nei suoi plugin.

Eclipse si offre il plugin nell'Eclipse Marketplace.

Se vogliamo utilizzare Kotlin per sviluppare applicazioni Android non possiamo che avvalerci di IntelliJ Idea oppure di Android Studio che su di esso è basato.

* * *

I programmi Kotlin che produciamo con l'IDE, se l'unica nostra risorsa per programmare Kotlin è l'IDE, possiamo eseguirli richiamandoli nello stesso IDE e da lì lanciaarli. Salvo avere installato anche il compilatore e lanciaarli con il comando `kotlin` da terminale.

Per produrre un file java eseguibile `.jar` con IntelliJ Idea procediamo come qui descritto.

Sperimentato il risultato del nostro lavoro, da menu FILE scegliamo PROJECT STRUCTURE... e, dalla finestra di dialogo che compare, scegliamo ARTIFACTS.

Clicchiamo sul pulsante +, selezioniamo JAR e FROM MODULES WITH DEPENDENCIES....

Nel campo MAIN CLASS, clicchiamo sull'icona di ricerca e selezioniamo la classe principale del nostro progetto (se abbiamo un file Kotlin, ad esempio `Main.kt`, verrà mostrata la classe generata come `MainKt`).

Clicchiamo su OK e poi ancora su OK.

Nel menu in alto, andiamo su BUILD ► BUILD ARTIFACTS...e clicchiamo sul nome che ci viene proposto per il nostro file `jar` e selezioniamo BUILD.

Troviamo il file `.jar` compilato nella cartella `OUT/ARTIFACTS` del progetto.